SICOPH

Sistema de Costeo de Procedimientos Hospitalarios

Costeo quirúrgico por TDABC · Multi-hospital

DOCUMENTACIÓN TÉCNICA DEL APLICATIVO

Arquitectura · Stack tecnológico · Requerimientos · Módulos · Base de datos · Archivos principales

Proyecto	app-costeo (SICOPH) — repositorio `github.com/ZapataMor/app-costeo`
Marco académico	Tesis doctoral *Sistema de Gestión del Conocimiento para la Optimización de los Costos del Servicio de Cirugía de los Hospitales de Mediana Complejidad de La Guajira* — Prof. Orlando Manuel Ruiz Pérez, Universidad de La Guajira
Desarrollador	Luis Felipe Zapata Mora
Stack	Laravel 13 · PHP 8.3 · Inertia.js 3 · React 19 · TypeScript · Tailwind 4 · MySQL 8
Versión del documento	1.0 · 2026-09-19 — refleja el código en la rama `main` (commit 0912a62)
Estado del código	299 pruebas automatizadas · 1.521 aserciones · PHPStan nivel 7 · CI en GitHub Actions

Índice de contenido

1. Introducción . . . 5
1.1 Propósito del documento . . . 5
1.2 Qué es SICOPH . . . 5
1.3 Convenciones . . . 5
2. Contexto del proyecto . . . 6
2.1 El problema . . . 6
2.2 La solución propuesta por la tesis . . . 6
2.3 Actores . . . 6
2.4 Datos de referencia del dominio . . . 7
3. Requerimientos . . . 8
3.1 Requerimientos funcionales . . . 9
3.2 Requerimientos no funcionales . . . 10
3.3 Restricciones y estándares . . . 10
4. Stack tecnológico . . . 11
4.1 Backend . . . 11
4.2 Frontend . . . 11
4.3 Herramientas de construcción y calidad . . . 12
4.4 Métricas del código . . . 12
5. Arquitectura . . . 13
5.1 Visión general . . . 13
5.2 Capas de la aplicación . . . 13
5.3 Multi-tenant: base de datos única con hospital_id . . . 13
5.4 Patrón de petición Inertia . . . 14
5.5 Flujo de datos del costeo . . . 14
5.6 Decisiones arquitectónicas registradas . . . 14
6. Módulos funcionales . . . 16
6.1 Parámetros del hospital . . . 16
6.2 Registro de cirugías . . . 16
6.3 Pacientes . . . 17
6.4 Costeo y análisis . . . 17
6.5 Dashboard consolidado, historial y digitadores . . . 17
6.6 Autenticación y ajustes . . . 17
6.7 Comandos de consola . . . 18
7. Motor de costeo TDABC . . . 19
7.1 Fórmulas . . . 19
7.2 Snapshot: la cirugía congela sus parámetros . . . 19

7.3 Costos indirectos (CIF): dos vías excluyentes . . . 19
7.4 Redondeo una sola vez y reparto por mayor resto . . . 20
7.5 Estructura del desglose (costos_cirugia.detalle) . . . 20
7.6 Detección de outliers y alertas de sobrecosto . . . 21
7.7 Indicadores (KpiService) . . . 21
8. Estructura de la base de datos . . . 22
8.1 Relaciones principales . . . 22
8.2 Diccionario de tablas . . . 23
8.2.1 hospitales . . . 23
8.2.2 users . . . 23
8.2.3 pacientes . . . 23
8.2.4 salas_operatorias . . . 24
8.2.5 procedimientos_quirurgicos . . . 24
8.2.6 recursos_humanos . . . 24
8.2.7 insumos . . . 25
8.2.8 equipos_medicos . . . 25
8.2.9 conceptos_costo_indirecto . . . 25
8.2.10 plantilla_personal / plantilla_insumos / plantilla_equipos . . . 26
8.2.11 cirugias . . . 26
8.2.12 cirugia_procedimiento . . . 26
8.2.13 miembros_equipo_quirurgico . . . 27
8.2.14 consumo_insumos . . . 27
8.2.15 cirugia_equipo_medico . . . 27
8.2.16 costos_cirugia . . . 27
8.2.17 cirugia_concepto_indirecto . . . 28
8.2.18 resultados_clinicos . . . 28
8.2.19 facturaciones . . . 28
8.2.20 alertas_sobrecosto . . . 29
8.2.21 registros_actividad . . . 29
8.3 Enumeraciones del dominio . . . 29
8.4 Datos de demostración . . . 30
9. Estructura del proyecto . . . 31
10. Archivos principales y su funcionalidad . . . 32
10.1 Modelos (app/Models) . . . 32
10.2 Servicios de dominio (app/Services) . . . 33
10.3 Capa HTTP (app/Http) . . . 34
10.4 Soporte, excepciones y proveedores . . . 35
10.5 Frontend (resources/js) . . . 36
11. Rutas y API . . . 37

11.1 Rutas web . . . 37
11.2 API v1 (JSON, sesión web) . . . 38
12. Seguridad, roles y protección de datos . . . 39
12.1 Autenticación . . . 39
12.2 Autorización por roles . . . 39
12.3 Aislamiento de datos . . . 39
12.4 Protección de datos personales (Ley 1581/2012) . . . 39
12.5 Integridad del costeo . . . 39
13. Calidad, pruebas e integración continua . . . 40
13.1 Suite de pruebas . . . 40
13.2 Análisis estático y formato . . . 41
13.3 Integración continua . . . 41
14. Instalación, configuración y despliegue . . . 42
14.1 Requisitos . . . 42
14.2 Puesta en marcha local . . . 42
14.3 Variables de entorno relevantes . . . 42
14.4 Despliegue en producción (lineamientos) . . . 42
14.5 Puesta en marcha del costeo en un hospital . . . 43
15. Estado actual, backlog y próximos pasos . . . 44
15.1 Estado al 2026-09-19 . . . 44
15.2 Backlog priorizado . . . 44
15.3 Próximos pasos . . . 44
Anexo A. Glosario . . . 46
Anexo B. Referencias del proyecto . . . 47

1. Introducción

1.1 Propósito del documento

Este documento describe de manera integral el aplicativo **SICOPH** (Sistema de Costeo de Procedimientos Hospitalarios): su contexto y objetivos, los requerimientos que atiende, el stack tecnológico, la arquitectura, los módulos funcionales, el motor de costeo, la estructura de la base de datos, la organización del código fuente con sus archivos principales, las rutas y la API, el esquema de seguridad, la estrategia de calidad y pruebas, el procedimiento de instalación y el estado actual del proyecto.

Está dirigido a desarrolladores que deban mantener o extender el sistema, al director del proyecto académico y al personal técnico de los hospitales que lo implanten. Toda la información fue extraída directamente del código fuente, las migraciones, las pruebas y la base de conocimiento del proyecto (`cerebro/`, `docs/`) a la fecha indicada en la portada.

1.2 Qué es SICOPH

SICOPH es un aplicativo web **multi-hospital** que captura los tiempos quirúrgicos, los consumos y los parámetros de costo de cada hospital, calcula el **costo real de cada cirugía** mediante la metodología **TDABC** (*Time-Driven Activity-Based Costing*), lo compara contra la tarifa facturada (referencia SOAT −25 %) y expone indicadores (KPIs bajo el marco Donabedian), dashboards, detección de outliers y alertas de sobrecosto.

Es la materialización tecnológica de la tesis doctoral del profesor Orlando Ruiz: el modelo de costeo ABC/TDABC que hoy vive en hojas de Excel se convierte en un sistema de información que captura datos estructurados (capa 2 de la arquitectura de Kerschberg), los transforma en información (capa 3) y sienta las bases de la gestión del conocimiento (capas 4 y 5).

1.3 Convenciones

- Los nombres de archivos, tablas, columnas, clases y rutas aparecen en `fuente monoespaciada`.
- Las cifras monetarias están en pesos colombianos (COP). Las fechas siguen el formato AAAA-MM-DD.
- Las rutas de archivos son relativas a la raíz del repositorio.
- Los comentarios del código, la interfaz de usuario y la documentación están en español, por decisión de proyecto.

2. Contexto del proyecto

2.1 El problema

Los hospitales públicos de mediana complejidad de La Guajira (E.S.E. Hospital San José de Maicao, Hospital Nuestra Señora de los Remedios de Riohacha y Hospital San Rafael de San Juan del Cesar) **no conocen cuánto les cuesta realmente cada cirugía**. Negocian tarifas con las EPS sin información de costos y en ocasiones a pérdida: el libro del profesor Ruiz (2022) demostró que el hospital de Maicao perdía hasta $94.310 por día de estancia en ciertas habitaciones sin saberlo.

Las hojas de Excel de recolección de costos quirúrgicos definen qué se captura por cirugía (equipo humano y sus tiempos, medicamentos, insumos, esterilización, ropa quirúrgica, quirófano por minuto, depreciación de equipos, personal indirecto, logística), pero no escalan a tres hospitales ni permiten análisis sistemático.

2.2 La solución propuesta por la tesis

Un **Sistema de Gestión del Conocimiento (SGC)** con arquitectura de cinco capas (Kerschberg) que capture datos quirúrgicos, los convierta en información (indicadores, dashboards) y en conocimiento (costos reales, variabilidad, mejores prácticas, lecciones aprendidas) con la meta de **reducir los costos quirúrgicos entre un 10 y un 20 %**.

Capa	Contenido según la tesis	Estado en SICOPH
1 — Objetos	Repositorio documental, protocolos, guías	No iniciada
2 — Datos	Captura estructurada multi-hospital con validaciones	**Completa**: modelo de datos, ciclo por fases, plantillas de protocolo, trazabilidad del dato
3 — Información / BI	Motor de costeo, indicadores, dashboards	**Completa**: motor TDABC + CIF por bolsas, outliers, KPIs Donabedian, 4 dashboards, explorador por procedimiento y por persona, exportación CSV
4 — Conocimiento	Reglas, alertas, lecciones aprendidas, comunidades de práctica	**Parcial**: alertas de sobrecosto con causa y revisión. Faltan lecciones aprendidas y recomendaciones
5 — Presentación	Portales por tipo de usuario	**Parcial**: tres roles con vistas propias. Faltan portal del cirujano, portal ejecutivo consolidado, portal público y puesta en producción

2.3 Actores

Actor	Descripción
Cliente / dueño de la visión	Prof. Orlando Manuel Ruiz Pérez — Universidad de La Guajira, doctorando, ex gerente del Hospital San José de Maicao.
Desarrollador	Luis Felipe Zapata Mora — desarrollador contratado del aplicativo completo.
Beneficiarios	Los tres hospitales públicos de mediana complejidad de La Guajira como punto de partida; el sistema es genérico para cualquier IPS.
Usuarios finales	Administradores de hospital (parámetros, costeo, análisis), digitadores (captura en sala/secretaría de quirófano) y super-administrador de la plataforma.

2.4 Datos de referencia del dominio

Dato	Valor
Caso de prueba obligatorio del motor	Cesárea por TDABC = **$520.000 COP** (capítulo 5 de la tesis)
Capacidad mensual por defecto	12 h/día × 26 días/mes × 60 min = 18.720 minutos disponibles
Referencia tarifaria	Tarifa SOAT −25 % (pactada con las EPS en Maicao)
Tasas del pipeline doctoral TDABC+MILP	Cirujano 1.550 COP/min · Quirófano 3.800 COP/min · CIF 420 COP/min
Procedimientos piloto	Cesáreas, apendicectomías, herniorrafias, colecistectomías, partos, legrados
Estándares obligatorios	CUPS (procedimientos), CIE-10 (diagnósticos), ATC (medicamentos), Ley 1581/2012 (datos personales)
Meta de reducción de costos	10–15 % (nivel 4 de madurez) → 15–20 % (nivel 5)

3. Requerimientos

Los requerimientos se reconstruyeron a partir del alcance confirmado con el cliente (2026-07-06), la Fase C (TOGAF) de la tesis, el capítulo 5 y el código implementado. Se indica el estado de cada uno.

3.1 Requerimientos funcionales

ID	Requerimiento	Módulo	Estado
RF-01	Registrar hospitales con sus parámetros de capacidad (horas/día, días/mes, minutos efectivos por hora) y factor de costos indirectos.	Parámetros	Implementado
RF-02	Administrar por hospital el catálogo de recursos humanos (rol, salario, prestaciones, indirectos), insumos (categoría, ATC, costo unitario), equipos médicos (costo/hora), salas operatorias (costo/hora) y procedimientos quirúrgicos (CUPS, duración, tarifa SOAT).	Parámetros	Implementado
RF-03	Registrar la trazabilidad de cada parámetro: fuente y nivel de confiabilidad (medido / estimado / supuesto).	Parámetros	Implementado
RF-04	Administrar bolsas de costo indirecto (CIF) por categoría, inductor de asignación y vigencia; activar y desactivar categorías sin doble conteo.	Parámetros / Costeo	Implementado
RF-05	Definir la plantilla de protocolo de cada procedimiento (personal, insumos, equipos y tiempos por fase) y sugerirla automáticamente desde el histórico.	Parámetros	Implementado
RF-06	Registrar pacientes con documento cifrado, régimen, asegurador y zona; alta rápida desde el registro de cirugía.	Pacientes	Implementado
RF-07	Registrar cirugías con procedimientos (CUPS, principal), diagnóstico CIE-10, sala, equipo quirúrgico con rol, fase y tiempos, consumos de insumos por fase y equipos médicos con minutos de uso.	Cirugías	Implementado
RF-08	Capturar el ciclo quirúrgico por fases con marcas de tiempo (ingreso, inicio, incisión, cierre, fin, salida de recuperación) y estados programada → en proceso → en recuperación → realizada / cancelada.	Cirugías	Implementado
RF-09	Prellenar cada registro con la plantilla del procedimiento y mostrar una estimación de costo en vivo.	Cirugías	Implementado
RF-10	Corregir cirugías registradas conservando las tarifas congeladas; cerrar cirugías abiertas.	Cirugías	Implementado
RF-11	Registrar la facturación (valor facturado, glosado, recaudado, tarifa de referencia) y el resultado clínico (complicaciones, estancia, reingreso, mortalidad).	Cirugías	Implementado
RF-12	Calcular el costo real de cada cirugía realizada por TDABC: recurso humano, sala, equipos, insumos, directo, indirecto y total, con desglose línea a línea y por fase.	Motor de costeo	Implementado
RF-13	Congelar en cada cirugía los parámetros con que se costea (snapshot) para que el costo histórico no cambie al actualizar el catálogo.	Motor de costeo	Implementado
RF-14	Detectar outliers de costo por procedimiento (z-score y Tukey/IQR).	Costeo	Implementado
RF-15	Generar alertas de sobrecosto por cirugía con atribución por componente, estado y causa revisable.	Costeo	Implementado
RF-16	Exponer KPIs Donabedian: costo promedio, coeficiente de variación, margen costo/tarifa, utilización de salas, glosas y recaudo, completitud, componentes, tendencia mensual.	Indicadores	Implementado
RF-17	Dashboards: resumen ejecutivo, componentes, outliers, rentabilidad, variabilidad; explorador por procedimiento y costeo por persona; filtros por periodo.	Dashboards	Implementado
RF-18	Exportar a CSV cirugías, indicadores y pendientes respetando los filtros de la vista.	Exportación	Implementado
RF-19	Roles: super_admin (todos los hospitales), admin_hospital (su hospital), digitador (solo registra y corrige lo suyo). Gestión de digitadores y desactivación de usuarios.	Seguridad	Implementado
RF-20	Historial de auditoría de creaciones, actualizaciones y eliminaciones con usuario, hospital, IP y fecha.	Auditoría	Implementado
RF-21	Autenticación con verificación de correo, 2FA y passkeys; gestión de perfil y seguridad.	Seguridad	Implementado

ID	Requerimiento	Módulo	Estado
RF-22	Reporte de capacidad no utilizada, costo estándar vs. real, horas extra, ecuaciones de tiempo, lecciones aprendidas, portal del cirujano, captura offline en tablet, integración HIS/HCE.	Varios	Backlog (ver §15)

3.2 Requerimientos no funcionales

ID	Requerimiento	Cómo se cumple
RNF-01	**Multi-tenant**: aislamiento total de datos entre hospitales en una sola base de datos.	Columna `hospital_id` en toda tabla de dominio, trait `BelongsToHospital` + scope global `HospitalScope`, reglas `exists` acotadas al hospital, pruebas `TenantIsolationTest`.
RNF-02	**Protección de datos personales** (Ley 1581/2012).	Documento del paciente cifrado (cast `encrypted`) con hash SHA-256 para unicidad y búsqueda; los KPIs solo devuelven agregados.
RNF-03	**Reproducibilidad y auditabilidad del costo**.	Snapshot de tarifas y parámetros CIF en la cirugía; desglose JSON en `costos_cirugia.detalle`; pivote `cirugia_concepto_indirecto`; historial de actividad.
RNF-04	**Exactitud contable**: los desgloses deben cuadrar al centavo con el total.	Redondeo una sola vez y reparto por mayor resto (`AsignadorCif::repartirPorMayorResto`); pruebas que fijan Σ líneas = total.
RNF-05	**Integridad**: consistencia temporal, códigos estándar, referencias válidas.	Form Requests con reglas de fecha/hora, regex CIE-10, CUPS de 6 dígitos, ATC obligatorio en medicamentos; claves foráneas con `restrictOnDelete`.
RNF-06	**Calidad de código**.	PHPStan/Larastan nivel 7, Pint, ESLint, Prettier, TypeScript estricto; CI en GitHub Actions con matriz PHP 8.3/8.4/8.5.
RNF-07	**Pruebas automatizadas** con el caso de referencia de la tesis.	299 pruebas (unitarias y de característica); la cesárea de $520.000 es prueba obligatoria.
RNF-08	**Sostenibilidad y bajo costo de operación** (principio 7 de la tesis).	Stack de código abierto (Laravel, React, MySQL); una sola base de datos; sin dependencias de pago.
RNF-09	**Usabilidad** para personal de sala con alta rotación.	Interfaz en español, registro prellenado por plantilla, estimación de costo en vivo, filtros comunes, modo claro/oscuro, diseño responsivo.
RNF-10	**Portabilidad de la base de datos**.	Migraciones compatibles con MySQL 8 (producción) y SQLite (pruebas en memoria).

3.3 Restricciones y estándares

- **CUPS** para procedimientos (código de 6 dígitos, único por hospital), **CIE-10** para diagnósticos (patrón `^[A-Z]\d{2}(\.\d{1,2})?$`), **ATC** obligatorio para insumos de categoría medicamento.
- Montos en `DECIMAL(14,2)` COP; tasas y porcentajes con mayor precisión (`DECIMAL(16,6)` para tasas CIF).
- Contexto real de operación: conectividad inestable, equipos limitados, alta rotación de personal (motiva la captura prellenada y la futura captura offline).
- El repositorio remoto es público: no se versionan datos personales de pacientes ni credenciales reales; los seeders son ficticios y están marcados `[SEMILLA]`.

4. Stack tecnológico

El stack fue decidido el 2026-07-06 (Laravel + MySQL + React, el stack del desarrollador) partiendo del *starter kit* oficial de Laravel para React, lo que aporta autenticación completa (Fortify), Inertia, Tailwind y componentes UI base.

4.1 Backend

Componente	Versión	Uso en el proyecto
PHP	≥ 8.3 (CI: 8.3, 8.4, 8.5)	Lenguaje del servidor. Se usan enums tipados, propiedades promovidas, *readonly* y tipado estricto.
Laravel	13.x (`laravel/framework ^13.17`)	Framework: enrutamiento, Eloquent ORM, migraciones, validación (Form Requests), colas, sesiones, gates.
Inertia.js (servidor)	`inertiajs/inertia-laravel ^3.0`	Puente entre controladores Laravel y páginas React sin API REST intermedia.
Laravel Fortify	^1.37	Autenticación: login, registro, verificación de correo, recuperación de contraseña, 2FA (TOTP), passkeys (WebAuthn).
Laravel Wayfinder	^0.1.14	Genera funciones TypeScript tipadas para cada ruta y controlador (rutas seguras en el frontend).
Laravel Chisel / Tinker / Pail	—	Herramientas de desarrollo: REPL, visor de logs.
MySQL	8.x	Base de datos de desarrollo local y producción.
SQLite	en memoria	Solo para la suite de pruebas (`phpunit.xml`).

4.2 Frontend

Componente	Versión	Uso en el proyecto
React	19.2 (con React Compiler vía Babel)	Biblioteca de interfaz. Páginas en `resources/js/pages`.
Inertia.js (cliente)	`@inertiajs/react ^3.0`	Navegación SPA, formularios (`useForm`), props compartidas.
TypeScript	^5.7	Tipado estricto de páginas, componentes y contratos de datos (`resources/js/types`).
Tailwind CSS	4.x (`@tailwindcss/vite`)	Estilos utilitarios; `tw-animate-css`, `tailwind-merge`, `class-variance-authority`.
Radix UI	Avatar, Checkbox, Collapsible, Dialog, Dropdown, Label, Navigation, Popover, Select, Separator, Toggle, Tooltip	Primitivas accesibles sobre las que se construyen los componentes de `components/ui` (estilo shadcn/ui).
Recharts	^3.9	Gráficas de los dashboards: barras apiladas, dispersión, boxplot, tendencia.
lucide-react	^0.475	Iconografía.
sonner	^2.0	Notificaciones tipo *toast* (mensajes flash del servidor).
cmdk · input-otp · @laravel/passkeys	—	Buscador tipo *command palette*, códigos 2FA, registro/uso de passkeys.

4.3 Herramientas de construcción y calidad

Herramienta	Función
Vite 8	Empaquetado y HMR del frontend; plugins `laravel-vite-plugin`, `@vitejs/plugin-react`, `@laravel/vite-plugin-wayfinder`.
PHPUnit 12	Pruebas unitarias y de característica (299 pruebas).
PHPStan / Larastan	Análisis estático nivel 7 sobre `app/`, `bootstrap/app.php`, `config/`, `database/`.
Laravel Pint	Formato de código PHP (PSR-12 / estilo Laravel).
ESLint 9 + Prettier 3	Lint y formato del frontend (plugins React, React Hooks, import, Tailwind).
GitHub Actions	Workflows `lint.yml` (Pint, Prettier, ESLint) y `tests.yml` (matriz PHP 8.3/8.4/8.5, build de Vite y `php artisan test`).
Composer scripts	`composer setup`, `composer dev` (servidor + cola + Vite), `composer lint`, `composer types:check`, `composer test`, `composer ci:check`.

4.4 Métricas del código

Métrica	Valor
Líneas de PHP en `app/`	≈ 11.900
Líneas de TypeScript/TSX en `resources/js/`	≈ 46.100 (incluye rutas generadas por Wayfinder)
Migraciones	35 (5 del framework + 30 de dominio)
Tablas de dominio	24 (más `users`, `passkeys`, caché, colas)
Modelos Eloquent	22
Controladores	31 (web + API)
Form Requests	30
Servicios de dominio	12
Páginas Inertia (React)	54
Pruebas / aserciones	299 / 1.521
Historial de Git	42 commits desde 2026-07-06

5. Arquitectura

5.1 Visión general

SICOPH es un **monolito modular** Laravel con frontend React renderizado a través de Inertia.js. No existe una API REST separada para la interfaz: los controladores devuelven páginas Inertia con sus *props* y el cliente React las renderiza. Una pequeña API JSON (`/api/v1`) expone los KPIs y la captura para consumo programático y para las pruebas.

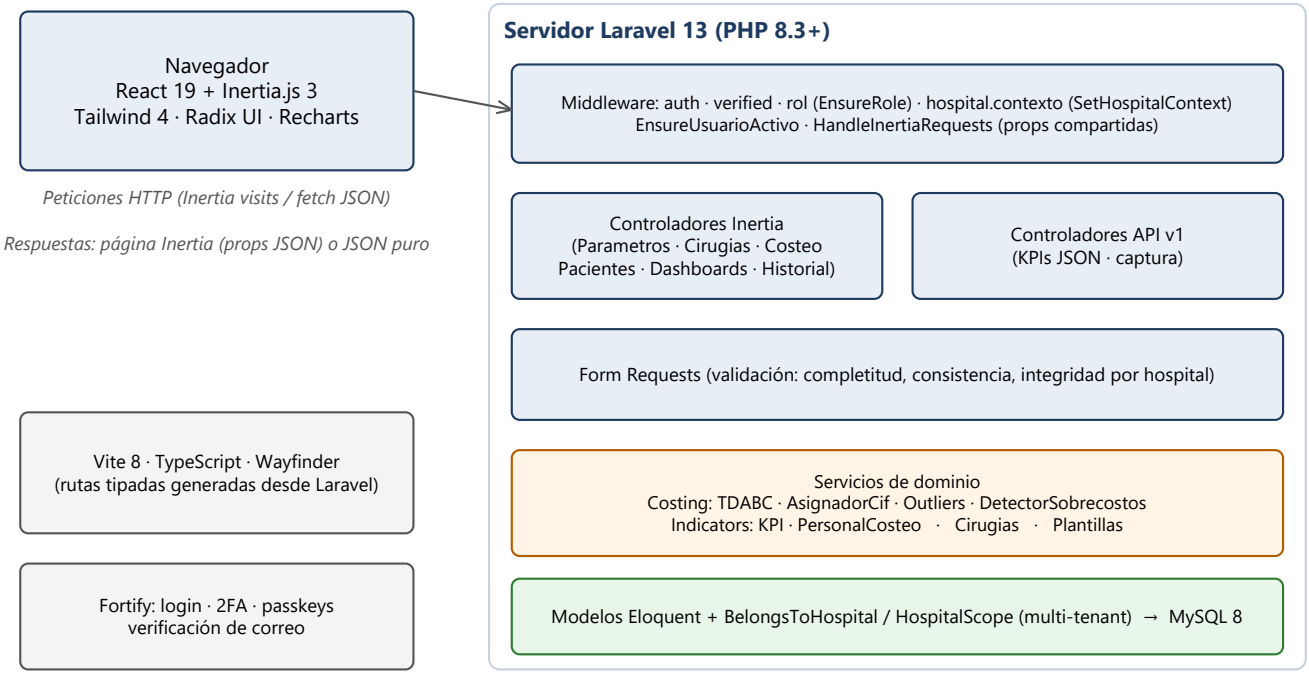


Figura 1. Arquitectura lógica del aplicativo.

5.2 Capas de la aplicación

Capa	Responsabilidad	Ubicación
Presentación	Páginas React, componentes, layouts, hooks y utilidades de formato. Recibe props tipadas desde Inertia.	`resources/js/`
HTTP	Rutas, middleware, controladores (delgados) y Form Requests (validación). Los controladores orquestan servicios y devuelven Inertia o JSON.	`routes/`, `app/Http/`
Dominio / servicios	Reglas de negocio: registro transaccional de cirugías, motor TDABC, asignación CIF, detección de outliers y sobrecostos, KPIs, plantillas.	`app/Services/`
Modelo	Entidades Eloquent, relaciones, casts, enums, scopes multi-tenant y auditoría.	`app/Models/`, `app/Enums/`
Soporte	Utilidades transversales: contexto de hospital, estadística, periodos, exportación CSV, excepciones de dominio.	`app/Support/`, `app/Exceptions/`
Persistencia	Migraciones, factories y seeders; MySQL 8.	`database/`

5.3 Multi-tenant: base de datos única con hospital_id

Cada tabla de dominio referencia `hospitales.id`. El trait `BelongsToHospital` (`app/Models/Concerns`) aplica el scope global `HospitalScope` que **filtra toda consulta Eloquent** por el hospital activo y lo **asigna automáticamente al crear** registros. El hospital activo se resuelve en `App\Support\HospitalContext`: por defecto es el del usuario autenticado (`users.hospital_id`); el middleware `SetHospitalContext` lo fija en cada petición y el `super_admin` puede cambiarlo con el *switcher* (sesión `hospital_activo_id`).

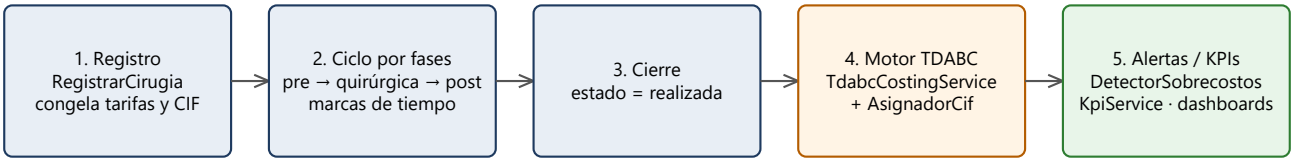
- El *route-model-binding* hereda el scope: un recurso de otro hospital responde **404** sin código adicional.
- Las reglas `exists` de los Form Requests se acotan al hospital: no se puede referenciar pacientes, salas o insumos de otro tenant.
- Las consultas SQL crudas (agregados de `PersonalCosteoService`, exportaciones) **deben filtrar por hospital explícitamente**: el scope solo protege a Eloquent.
- El `admin_hospital` queda siempre atado a su hospital (*fail-closed*: sin hospital asignado la petición se rechaza). El `super_admin` opera sin scope hasta que selecciona un hospital.

5.4 Patrón de petición Inertia

```
Navegador (React)  --Inertia visit-->  routes/web.php  -->  middleware (auth, verified, rol, hospital.contexto)
    -->  Controlador  -->  FormRequest (valida)  -->  Servicio de dominio (transacción)
    -->  Inertia::render('pagina', props)  -->  JSON de props  -->  React renderiza la página
```

`HandleInertiaRequests` comparte en todas las páginas el usuario autenticado, su rol, el hospital activo, la lista de hospitales (para el switcher del super_admin) y los mensajes *flash* que `use-flash-toast` convierte en notificaciones.

5.5 Flujo de datos del costeo



Cada cirugía se costea con SU foto de parámetros (snapshot), nunca con el catálogo vigente.

Figura 2. Ciclo de vida de una cirugía desde el registro hasta los indicadores.

5.6 Decisiones arquitectónicas registradas

El proyecto mantiene una nota por decisión en `cerebro/decisiones/` con el formato contexto → decisión → por qué → consecuencias. Las principales:

Fecha	Decisión	Resumen
2026-07-06	Stack Laravel + Inertia + React	Starter kit oficial; un solo despliegue; rutas tipadas con Wayfinder.
2026-07-06	Multi-tenant con BD única y hospital_id	Scope global + trait; aislamiento verificable por pruebas; permite comparar hospitales a futuro.
2026-07-06	Cifrado del documento del paciente (Ley 1581)	Cast `encrypted` + `documento_hash` SHA-256 único por hospital.
2026-07-06	Cesárea de $520.000 como caso de prueba	Prueba obligatoria que fija la equivalencia con la tesis.
2026-07-11	Tres roles: super_admin, admin_hospital, digitador	Modelo mínimo de acceso; el digitador solo registra y corrige lo suyo.
2026-07-13	Snapshot de parámetros en la cirugía	El motor lee la foto congelada, no el catálogo vigente; el costo es un hecho histórico.
2026-07-21	Ciclo por fases y plantillas de protocolo	Marcas de tiempo, estados, fase por recurso/consumo, plantillas prellenadas y sugeridas desde el histórico.
2026-09-03	Capacidad práctica configurable	`minutos_efectivos_hora` por hospital (Kaplan & Anderson ≈ 80 %).
2026-09-04	CIF: bolsas vs. factor, vías excluyentes	ABC por inductor cuando hay bolsas activas; factor plano en caso contrario; nunca ambas.

Fecha	Decisión	Resumen
2026-09-04	Anti-doble-conteo: marcar origen	Activar una categoría CIF marca el componente directo equivalente como derivado, sin borrar el dato.
2026-09-06	Denominador = capacidad sumada del hospital	Las tasas por minuto se calculan sobre todas las salas / todo el personal, no sobre un recurso.
2026-09-06	Redondeo una vez y mayor resto	Los desgloses cuadran al centavo con el total.
2026-09-15	El cerebro (vault Obsidian) vive en el repo	Conocimiento del proyecto versionado junto al código.

6. Módulos funcionales

Los módulos se corresponden con los prefijos de ruta, los directorios de controladores y las carpetas de páginas React. Cada uno se describe con su propósito, sus funcionalidades, los roles que acceden y sus archivos clave.

6.1 Parámetros del hospital

Ruta `/parametros` · **Roles** super_admin, admin_hospital · **Controladores** `app/Http/Controllers/Parametros/*` · **Páginas** `resources/js/pages/parametros/*`

Es el catálogo de cada hospital: lo que el motor de costeo necesita saber antes de costear la primera cirugía. Cada parámetro lleva **trazabilidad** (`fuente`, `nivel_confiabilidad`: medido / estimado / supuesto) para que la calidad del dato sea visible.

Submódulo	Funcionalidad	Controlador
Configuración del hospital	Horas/día, días/mes, minutos efectivos por hora (capacidad práctica), factor de costos indirectos. Advierte cuando hay bolsas CIF activas (el factor se ignora) o cuando no hay ningún indirecto configurado.	HospitalConfiguracionController
Recursos humanos	CRUD del personal quirúrgico: nombre, rol (cirujano, ayudante, anestesiólogo, instrumentador, circulante), especialidad, salario, prestaciones e indirectos mensuales, activo.	RecursoHumanoController
Insumos	CRUD de medicamentos, dispositivos y materiales: código, ATC (obligatorio en medicamentos), unidad, costo unitario.	InsumoController
Equipos médicos	CRUD: nombre, código, valor de adquisición, vida útil, costo/hora.	EquipoMedicoController
Salas operatorias	CRUD: nombre, ubicación, costo/hora, equipamiento (JSON), activa.	SalaOperatoriaController
Procedimientos quirúrgicos	CRUD: código CUPS, nombre, especialidad, complejidad, duración estimada, tiempos por fase (prequirúrgico, recuperación, recambio), tarifa SOAT.	ProcedimientoQuirurgicoController
Plantilla del protocolo	Por procedimiento: personal por rol y fase, insumos por fase con cantidad, equipos con minutos; líneas opcionales. Botón *Sugerir* deduce la plantilla del histórico (`GeneradorPlantilla`).	PlantillaProcedimientoController
Costos indirectos (bolsas CIF)	CRUD de conceptos: nombre, categoría, base de asignación, monto mensual o porcentaje, vigencia. Panel de activación/desactivación por categoría con diálogo que nombra los conflictos de doble conteo.	ConceptoCostoIndirectoController

6.2 Registro de cirugías

Ruta `/cirugias` · **Roles** todos (el digitador solo ve y corrige lo que él capturó) · **Controladores** `app/Http/Controllers/Cirugias/*` · **Páginas** `resources/js/pages/cirugias/*`

Es el módulo de captura (capa 2). El registro es **transaccional**: cirugía + procedimientos + equipo quirúrgico + consumos + equipos médicos se guardan juntos o no se guardan, y en ese momento se **congelan** todas las tarifas vigentes.

- **Inicio** (`cirugias/inicio.tsx`): pantalla del digitador con accesos rápidos y sus registros recientes.
- **Listado** con filtros comunes (`FiltraListado`): texto, procedimiento, estado, rango de fechas; exportación CSV.
- **Nuevo registro** (`formulario-cirugia.tsx`): selección de paciente con alta rápida en modal, procedimientos con principal, sala, marcas de tiempo del ciclo (ingreso, inicio, incisión, cierre, fin, salida de recuperación), equipo por rol y fase con horas de entrada/salida (minutos derivados), consumos por fase, equipos médicos. Se prellena con la plantilla del procedimiento y muestra una **estimación de costo** en vivo (`estimacion-costo.tsx`).
- **Corrección** (`ActualizarCirugia`): conserva las tarifas congeladas de lo que ya existía; solo lo nuevo toma la tarifa vigente. **Cierre** de una cirugía abierta (`cerrar-cirugia-modal.tsx`): registra la marca que falta y dispara el costeo.

- **Detalle** (`cirugias/show.tsx`, solo administradores): ficha completa con desglose del costo por componente, fase y bolsa CIF (`desglose-costo.tsx`), tarjeta de facturación y de resultado clínico.
- **Estados**: programada → en_proceso → en_recuperacion → realizada | cancelada. Solo una cirugía **realizada** se costea.

6.3 Pacientes

Ruta `/pacientes` · **Roles** super_admin, admin_hospital (el digitador solo hace alta rápida desde el registro). Padrón de pacientes del hospital con documento cifrado, búsqueda por hash, edición en modal y eliminación protegida (un paciente con cirugías no se puede borrar por la FK `restrictOnDelete`).

6.4 Costeo y análisis

Ruta `/costeo` · **Roles** super_admin, admin_hospital · **Controladores** `DashboardCosteoController`, `app/Http/Controllers/Costeo/*` · **Páginas** `resources/js/pages/costeo/*`

Vista	Ruta	Contenido
Resumen ejecutivo	/costeo	KPIs principales (costo promedio, margen, utilización, glosas), completitud de captura, tendencia mensual del costo, selector de periodo.
Componentes	/costeo/componentes	Barras apiladas del costo promedio por componente TDABC (recurso humano, sala, equipos, insumos, indirecto) por procedimiento.
Outliers	/costeo/outliers	Dispersión de costos por procedimiento con límites estadísticos (z-score y Tukey) y boxplot.
Rentabilidad	/costeo/rentabilidad	Costo real vs. tarifa facturada y vs. tarifa SOAT −25 %; margen por procedimiento.
Variabilidad	/costeo/variabilidad	Coeficiente de variación por procedimiento con umbrales 15 % / 30 %.
Explorador de procedimientos	/costeo/procedimientos	Catálogo → cirugías del procedimiento → detalle costeado de cada cirugía (composición, comparación con la mediana).
Costeo por persona	/costeo/personal	Ranking del personal por costo propio (sus minutos) y costo inducido (lo que moviliza como cirujano) con índice ajustado por case-mix; ficha individual con desgloses, por procedimiento e historial paginado.
Bandeja de alertas	/costeo/alertas	Alertas de sobrecosto pendientes / revisadas / descartadas; revisión con causa (consumo excesivo, tiempo prolongado, complicación, error de registro…) y detalle.

6.5 Dashboard consolidado, historial y digitadores

- **Dashboard** (`/dashboard`, solo super_admin): vista consolidada multi-hospital con el *switcher* de hospital activo (`HospitalActivoController`).
- **Historial** (`/historial`): bitácora de auditoría (`registros_actividad`) generada por el trait `Auditable`: acción, descripción, usuario, hospital, entidad afectada, IP y fecha.
- **Digitadores** (`/digitadores`, solo admin_hospital): creación de usuarios con rol digitador para su hospital y activación/desactivación (`EnsureUsuarioActivo` cierra la sesión de un usuario desactivado).
- **Exportaciones** (`/exportar/{cirugias|indicadores|pendientes}`): CSV en streaming (`ExportadorCsv`) que respeta los filtros de la vista de origen.

6.6 Autenticación y ajustes

Provisto por Fortify y el starter kit: login, registro, verificación de correo, recuperación de contraseña, confirmación de contraseña, desafío 2FA (`pages/auth/*`); perfil, contraseña, 2FA con códigos de recuperación, passkeys y apariencia (`pages/settings/*`, `routes/settings.php`).

6.7 Comandos de consola

Comando	Función
`php artisan costeo:detectar-sobrecostos {--hospital=}`	Barre las cirugías ya costeadas y genera alertas de sobrecosto (útil tras cargar histórico o cambiar umbrales).
`php artisan plantillas:generar`	Deduce la plantilla estándar de cada procedimiento desde sus cirugías registradas.
`php artisan migrate:fresh --seed`	Reconstruye el esquema y siembra dos hospitales ficticios con la cesárea de referencia de $520.000.

7. Motor de costeo TDABC

El corazón del sistema es `App\Services\Costing\TdabcCostingService`. Recibe una cirugía en estado **realizada**, lee los parámetros congelados en ella y produce un registro `costos_cirugia` con el desglose completo. Al terminar, invoca al `DetectorSobrecostos`.

7.1 Fórmulas

```
costo total        = costo directo + costo indirecto
costo directo      = costo recurso humano + costo sala + costo equipos + costo insumos

costo RRHH         = Σ ( costo_mensual_registrado × minutos_participacion ÷ minutos_disponibles_mes )
costo_mensual      = salario_mensual + prestaciones_mensuales + costos_indirectos_mensuales*
minutos_disp./mes  = horas_dia × dias_mes × minutos_efectivos_hora        (12 × 26 × 60 = 18.720)

costo sala         = costo_hora_sala_registrado × duración(hora_fin − hora_inicio) ÷ 60
costo equipos      = Σ ( costo_hora_registrado × minutos_uso ÷ 60 )
costo insumos      = Σ ( cantidad × costo_unitario_registrado )

costo indirecto    = vía 'factor':  costo directo × factor_indirecto_registrado
                   = vía 'bolsas':  Σ bolsa_i  donde  bolsa_i = tasa_i × unidades_i
      tasa_i       = monto_mensual_i ÷ denominador_i   (minuto_quirofano | minuto_personal)
                   = porcentaje_i                       (porcentaje_directo)

* se excluye cuando la categoría CIF personal_indirecto está activa (evita doble conteo)
```

Caso de referencia (prueba obligatoria `tests/Unit/TdabcCostingServiceTest.php`): cesárea = cirujano $50.000/h × 1,5 h + ayudante $30.000 × 1,5 + anestesiólogo $50.000 × 2 + instrumentador $20.000 × 2 + circulante $15.000 × 2 + sala $40.000 × 2 + insumos $150.000 = **$520.000 COP**.

7.2 Snapshot: la cirugía congela sus parámetros

`RegistrarCirugia` copia, al registrar, los valores vigentes del catálogo a la cirugía y a sus pivotes. El motor lee esa foto, nunca el catálogo actual; las tarifas vigentes solo se usan de respaldo para datos anteriores al snapshot.

Columna congelada	Tabla	Origen
`minutos_disponibles_mes_registrado`	`cirugias`	Hospital::minutosDisponiblesMes()
`minutos_efectivos_hora_registrado`	`cirugias`	hospitales.minutos_efectivos_hora
`factor_indirecto_registrado`	`cirugias`	hospitales.factor_indirecto
`costo_hora_sala_registrado`	`cirugias`	salas_operatorias.costo_hora
`parametros_cif_registrados (JSON)`	`cirugias`	AsignadorCif::congelar(): vía, orígenes, denominadores, bolsas con tasa
`costo_mensual_registrado`	`miembros_equipo_quirurgico`	RecursoHumano::costoMensualTotal(sinIndirectos?)
`costo_hora_registrado`	`cirugia_equipo_medico`	equipos_medicos.costo_hora
`costo_unitario_registrado, costo_total`	`consumo_insumos`	insumos.costo_unitario × cantidad

7.3 Costos indirectos (CIF): dos vías excluyentes

Vía	Cuándo aplica	Fórmula
factor	El hospital no tiene bolsas CIF activas y vigentes a la fecha de la cirugía (o la cirugía es anterior al módulo CIF).	costo_directo × factor_indirecto
bolsas	Hay al menos una bolsa activa y vigente. Es el costeo ABC propiamente dicho.	Σ de cada bolsa repartida por su propio inductor

Bases de asignación (`BaseAsignacionCif`):

Base	Denominador (capacidad sumada del hospital)	Unidades de la cirugía
`minuto_quirofano`	salas activas × horas_dia × dias_mes × minutos_efectivos_hora	duración real en sala
`minuto_personal`	personal quirúrgico activo × misma capacidad	Σ minutos de participación del equipo
`porcentaje_directo`	—	costo directo × porcentaje

Categorías CIF (`CategoriaCif`) y el componente directo que reemplazan (invariante anti-doble-conteo):

Categoría	Componente directo que duplicaría	Qué hace activarla
`infraestructura`	`salas_operatorias.costo_hora`	Marca hospitales.origen_infraestructura = derivado_de_cif; el costo de sala pasa a 0 en el directo y entra por la bolsa.
`depreciacion_equipos`	`equipos_medicos.costo_hora`	Marca origen_depreciacion_equipos; el costo de equipos entra por la bolsa.
`personal_indirecto`	`recursos_humanos.costos_indirectos_mensuales`	Marca origen_personal_indirecto; el costo mensual congelado excluye los indirectos.
`administracion`	`ninguno`	Sin solape posible.
`servicios_generales`	`ninguno`	Sin solape posible.

Invariante: un componente de costo nunca puede estar contado por el catálogo directo y por una bolsa CIF a la vez, ni desaparecer de ambos. Se sostiene por construcción: `activo` está fuera del `$fillable` de `ConceptoCostoIndirecto` y las columnas `origen_*` fuera del de `Hospital`; `ActivarCategoriaCif` / `DesactivarCategoriaCif` son los únicos escritores y lo hacen en una sola transacción, exigiendo confirmación cuando el componente equivalente sigue digitado con valor > 0. El dato digitado se **marca**, no se borra, para poder comparar «método actual vs. método por bolsas».

7.4 Redondeo una sola vez y reparto por mayor resto

Cada bolsa se calcula a precisión completa; el total se redondea a centavos una vez; las líneas se cuadran contra ese total por **mayor resto** (`AsignadorCif::repartirPorMayorResto()`). El mismo mecanismo prorratea el indirecto entre las fases del ciclo (pesos = participación de cada fase en el costo directo). Las pruebas fijan que Σ `cirugia_concepto_indirecto.monto_asignado` = `costo_indirecto` y Σ `detalle.indirecto_por_fase` = `costo_indirecto`.

7.5 Estructura del desglose (costos_cirugia.detalle)

```
{
  "minutos_disponibles_mes": 18720, "minutos_efectivos_hora": 60,
  "recurso_humano": [ {recurso_humano_id, nombre, rol, fase, minutos, costo_por_minuto, costo} ],
  "sala":     {sala_operatoria_id, nombre, minutos, costo_hora, costo, derivado_de_cif},
  "equipos":  [ {equipo_medico_id, nombre, minutos, costo_hora, costo, derivado_de_cif} ],
  "insumos":  [ {insumo_id, fase, nombre, cantidad, costo_unitario, costo} ],
  "por_fase": {"pre": 0.0, "quirurgica": 0.0, "post": 0.0},
  "indirecto": {"via": "bolsas"|"factor", "factor_indirecto": null|0.12,
               "bolsas": [ {concepto_costo_indirecto_id, nombre, categoria, base_asignacion,
                            monto_mensual, porcentaje, denominador, tasa, unidades, monto} ]},
  "indirecto_por_fase": {"pre": 0.0, "quirurgica": 0.0, "post": 0.0}
}
```

7.6 Detección de outliers y alertas de sobrecosto

Servicio	Pregunta que responde	Criterios
OutlierDetector	¿Cuáles cirugías de este procedimiento se salieron del rango? (exploración en dashboard, ventana temporal opcional)	z-score |z| > 3 (requiere desviación > 0) y Tukey: fuera de [Q1 − 1,5·IQR, Q3 + 1,5·IQR], por procedimiento principal.
DetectorSobrecostos	La cirugía que se acaba de costear, ¿se salió por encima? Genera una `AlertaSobrecosto` con atribución por componente.	Mismos criterios pero solo por encima; baseline mínimo de 5 cirugías comparables; exceso mínimo del 10 % sobre la mediana (significancia económica además de estadística). Atribuye el exceso a recurso humano, sala, equipos, insumos o indirecto y nombra el componente dominante.

Las alertas tienen estado (`pendiente`, `revisada`, `descartada`) y causa (`CausaSobrecosto`: consumo excesivo de insumos, tiempo quirúrgico prolongado, retraso de alistamiento, equipo o personal adicional, complicación clínica, caso complejo justificado, error de registro, otra), lo que cierra el ciclo entre detectar y corregir y es el primer paso de la capa 4 (lecciones aprendidas).

7.7 Indicadores (KpiService)

KPI	Dimensión Donabedian	Cálculo
`costos()`	Proceso	Costo promedio por cirugía (global) y por procedimiento principal.
`variabilidad()`	Proceso	Coeficiente de variación (σ/μ) por procedimiento; niveles con umbrales 15 % / 30 %.
`margen()`	Resultado	Costo real vs. tarifa facturada y vs. referencia SOAT −25 %, por procedimiento.
`utilizacionSalas(mes)`	Estructura	Minutos operados ÷ minutos disponibles por sala (capacidad vigente del hospital).
`glosasRecaudo()`	Resultado	Tasa de glosas y de recaudo sobre lo facturado.
`completitud()`	Supervisión	% de cirugías realizadas con equipo, insumos, costo, resultado clínico y factura.
`costoPorComponente()`	Proceso	Costo promedio por componente TDABC por procedimiento.
`tendenciaMensual()`	Proceso	Serie mensual: costo promedio, volumen y gasto total.

Todos los indicadores admiten una ventana temporal (`Periodo`, `enPeriodo()`) y solo consideran cirugías **realizadas con hora de fin y costo calculado**. Los indicadores de utilización usan la capacidad vigente, no la congelada: cambiar `horas_dia` reescribe porcentajes históricos de ocupación (los costos no cambian).

8. Estructura de la base de datos

Esquema relacional en MySQL 8 (SQLite en memoria para pruebas), definido por 35 migraciones en `database/migrations/`. Todas las tablas de dominio incluyen `hospital_id` (FK a `hospitales`, `restrictOnDelete`) y `timestamps`. Los montos son `DECIMAL(14,2)` en COP.

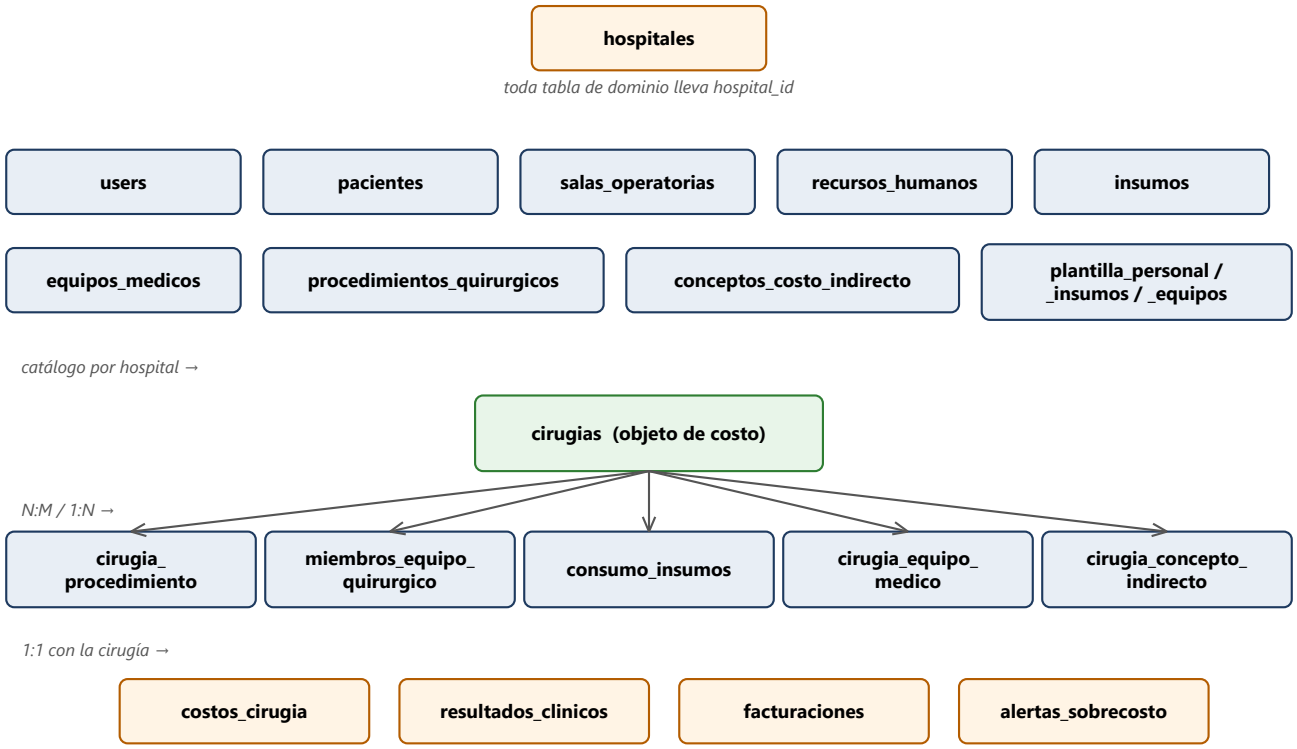


Figura 3. Modelo de datos: catálogo por hospital, la cirugía como objeto de costo, sus pivotes y sus resultados 1:1.

8.1 Relaciones principales

Relación	Cardinalidad	Tabla / clave
Hospital → usuarios, pacientes, salas, recursos humanos, insumos, equipos, procedimientos, conceptos CIF, cirugías	1:N	hospital_id en cada tabla
Paciente → cirugías	1:N	cirugias.paciente_id
Sala operatoria → cirugías	1:N (opcional)	cirugias.sala_operatoria_id
Usuario → cirugías registradas	1:N	cirugias.registrado_por
Cirugía ↔ procedimientos	N:M	cirugia_procedimiento (es_principal)
Cirugía → miembros del equipo ← recurso humano	1:N / N:1	miembros_equipo_quirurgico (rol, fase, minutos, costo congelado)
Cirugía → consumos ← insumo	1:N / N:1	consumo_insumos (fase, cantidad, costo congelado)
Cirugía ↔ equipos médicos	N:M	cirugia_equipo_medico (minutos_uso, costo_hora congelado)
Cirugía → líneas CIF ← concepto	1:N / N:1	cirugia_concepto_indirecto (tasa, unidades, monto)
Cirugía → costo / resultado clínico / facturación / alerta	1:1	cirugia_id UNIQUE en cada tabla
Procedimiento → plantilla de personal / insumos / equipos	1:N	plantilla_* (procedimiento_quirurgico_id)
Usuario / hospital → registros de actividad	1:N	registros_actividad (auditable morph)

8.2 Diccionario de tablas

8.2.1 hospitales

Entidad raíz del multi-tenant. Guarda la capacidad práctica y los parámetros globales de costeo del hospital.

Columna	Tipo	Descripción
id	bigint PK	Identificador.
nombre	varchar	Razón social.
nit	varchar(20) UNIQUE	NIT.
nivel_complejidad	varchar(20) = 'mediana'	baja / mediana / alta.
municipio, departamento	varchar	Ubicación (departamento por defecto 'La Guajira').
horas_dia	tinyint = 12	Horas de operación por día.
dias_mes	tinyint = 26	Días operativos por mes.
minutos_efectivos_hora	tinyint = 60	Capacidad práctica: minutos productivos por hora (Kaplan & Anderson ≈ 48).
factor_indirecto	decimal(5,4) = 0	Factor de costos indirectos sobre el directo (vía 'factor').
origen_infraestructura, origen_depreciacion_equipos, origen_personal_indirecto	varchar(20) = 'digitado'	digitado | derivado_de_cif. Solo los escribe ActivarCategoriaCif / DesactivarCategoriaCif.

8.2.2 users

Usuarios de la plataforma (tabla del framework extendida).

Columna	Tipo	Descripción
id, name, email, password	—	Campos estándar de Laravel; email verificado (email_verified_at).
hospital_id	FK hospitales NULL	Hospital al que pertenece (null para super_admin sin hospital).
role	varchar(30) = 'admin_hospital'	super_admin | admin_hospital | digitador (RolUsuario).
activo	boolean = true	Un usuario desactivado no puede iniciar sesión.
two_factor_secret, two_factor_recovery_codes, two_factor_confirmed_at	text / timestamp	2FA de Fortify.

Tablas relacionadas del framework: `passkeys` (WebAuthn), `sessions`, `password_reset_tokens`, `cache`, `jobs`, `failed_jobs`.

8.2.3 pacientes

Padrón de pacientes por hospital. El documento se cifra (Ley 1581/2012).

Columna	Tipo	Descripción
hospital_id	FK	Tenant.
tipo_documento	varchar(5) = 'CC'	Tipo de documento.
documento	text (cifrado)	Número de documento, cast `encrypted`.
documento_hash	varchar(64)	SHA-256 para búsqueda y unicidad; UNIQUE(hospital_id, documento_hash).
nombres, apellidos	varchar	Nombre.
fecha_nacimiento, sexo	date, char(1)	Datos demográficos.
regimen	varchar(20) = 'subsidiado'	contributivo | subsidiado | especial | no_asegurado.
asegurador, zona, municipio	varchar	EPS, urbana/rural, municipio.

8.2.4 salas_operatorias

Quirófanos del hospital con su tarifa por hora.

Columna	Tipo	Descripción
hospital_id	FK	Tenant.
nombre	varchar	UNIQUE(hospital_id, nombre).
ubicacion	varchar NULL	Piso / área.
costo_hora	decimal(14,2)	Tarifa por hora (depreciación + servicios + mantenimiento). Se excluye del directo si la categoría infraestructura está activa.
equipamiento	json NULL	Lista de equipamiento fijo.
activa	boolean = true	Cuenta para la capacidad de quirófanos.
fuente, nivel_confiabilidad	varchar	Trazabilidad del dato (medido / estimado / supuesto).

8.2.5 procedimientos_quirurgicos

Catálogo CUPS del hospital con duración y tiempos por fase.

Columna	Tipo	Descripción
hospital_id	FK	Tenant.
codigo_cups	varchar(10)	UNIQUE(hospital_id, codigo_cups); 6 dígitos.
nombre, especialidad	varchar	Descripción.
complejidad	varchar(10) = 'media'	baja | media | alta.
duracion_estimada_minutos	smallint	Duración planificada del acto quirúrgico.
minutos_prequirurgico, minutos_recuperacion, minutos_recambio	smallint NULL	Tiempos por fase de la plantilla.
tarifa_soat	decimal(14,2) NULL	Tarifa de referencia.
fuente, nivel_confiabilidad	varchar	Trazabilidad.

8.2.6 recursos_humanos

Personal quirúrgico con su costo mensual.

Columna	Tipo	Descripción
hospital_id	FK	Tenant; índice (hospital_id, rol).
nombre, especialidad	varchar	Identificación.
rol	varchar(20)	cirujano | ayudante | anestesiologo | instrumentador | circulante.
salario_mensual	decimal(14,2)	Salario.
prestaciones_mensuales	decimal(14,2) = 0	Prestaciones sociales.
costos_indirectos_mensuales	decimal(14,2) = 0	Indirectos del personal; se excluyen si la categoría personal_indirecto está activa.
activo	boolean = true	Cuenta para la capacidad de personal.
fuente, nivel_confiabilidad	varchar	Trazabilidad.

8.2.7 insumos

Medicamentos, dispositivos y materiales.

Columna	Tipo	Descripción
hospital_id	FK	Tenant.
codigo	varchar(30)	UNIQUE(hospital_id, codigo).
nombre	varchar	Descripción.
categoria	varchar(15)	medicamento | dispositivo | material.
codigo_atc	varchar(10) NULL	Obligatorio si categoria = medicamento.
unidad	varchar(20)	Unidad de medida.
costo_unitario	decimal(14,2)	Precio vigente (se congela en cada consumo).
activo, fuente, nivel_confiabilidad	–	Estado y trazabilidad.

8.2.8 equipos_medicos

Equipos con tarifa por hora de uso.

Columna	Tipo	Descripción
hospital_id	FK	Tenant.
nombre, codigo	varchar	Identificación.
valor_adquisicion, vida_util_anios	decimal, tinyint NULL	Base de depreciación.
costo_hora	decimal(14,2)	Tarifa por hora; se excluye del directo si depreciacion_equipos está activa.
activo, fuente, nivel_confiabilidad	–	Estado y trazabilidad.

8.2.9 conceptos_costo_indirecto

Bolsas CIF del hospital: monto mensual por categoría, inductor y vigencia.

Columna	Tipo	Descripción
hospital_id	FK	Tenant; índices por (hospital, categoría, vigencia) y (hospital, nombre).
nombre	varchar	Identidad del concepto = nombre + categoría.
categoria	varchar(30)	infraestructura | depreciacion_equipos | personal_indirecto | administracion | servicios_generales.
base_asignacion	varchar(30)	minuto_quirofano | minuto_personal | porcentaje_directo.
monto_mensual	decimal(14,2) NULL	Para bases por minuto.
porcentaje	decimal(8,4) NULL	Para porcentaje_directo.
vigente_desde, vigente_hasta	date, date NULL	Vigencias del mismo concepto no pueden solaparse.
activo	boolean = false	Fuera de $fillable; solo ActivarCategoriaCif lo escribe.
fuente, nivel_confiabilidad	varchar	Trazabilidad.

8.2.10 plantilla_personal / plantilla_insumos / plantilla_equipos

Plantilla del protocolo de cada procedimiento: con qué nace prellenado cada registro.

Columna	Tipo	Descripción
procedimiento_quirurgico_id	FK cascade	Procedimiento dueño de la plantilla.
plantilla_personal: rol, fase, cantidad, recurso_humano_id NULL, minutos NULL, opcional	—	Rol requerido por fase (persona concreta opcional).
plantilla_insumos: insumo_id, fase, cantidad, opcional	—	UNIQUE(procedimiento, insumo, fase).
plantilla_equipos: equipo_medico_id, minutos_uso NULL, opcional	—	UNIQUE(procedimiento, equipo).

8.2.11 cirugias

El evento quirúrgico: objeto de costo del modelo TDABC. Lleva las marcas de tiempo del ciclo y el snapshot de parámetros.

Columna	Tipo	Descripción
hospital_id, paciente_id, sala_operatoria_id NULL, registrado_por NULL	FK	Tenant, paciente, sala y usuario que capturó (base del gate corregir-cirugia).
fecha	date	Fecha de la cirugía; índice (hospital_id, fecha).
hora_ingreso_paciente, hora_inicio, hora_incision, hora_cierre, hora_fin, hora_salida_recuperacion	datetime (varios NULL)	Marcas del ciclo: pre (ingreso→inicio), quirúrgica (inicio→fin; neto incisión→cierre), post (fin→salida).
tipo	varchar(15) = 'programada'	programada | urgencia.
estado	varchar(15) = 'en_proceso'	programada | en_proceso | en_recuperacion | realizada | cancelada.
diagnostico_cie10	varchar(8) NULL	Código CIE-10.
observaciones	text NULL	Notas.
minutos_disponibles_mes_registrado, minutos_efectivos_hora_registrado, factor_indirecto_registrado, costo_hora_sala_registrado	int / tinyint / decimal NULL	Snapshot de parámetros del hospital y la sala.
parametros_cif_registrados	json NULL	Snapshot CIF: vía, orígenes, denominadores y bolsas con tasa. NULL = cirugía anterior al módulo (vía factor).

8.2.12 cirugia_procedimiento

Procedimientos realizados en la cirugía (N:M).

Columna	Tipo	Descripción
cirugia_id, procedimiento_quirurgico_id	FK (cascade / restrict)	UNIQUE por par.
es_principal	boolean	El principal agrupa los KPIs y el baseline de alertas.

8.2.13 miembros_equipo_quirurgico

Participación de cada persona en la cirugía.

Columna	Tipo	Descripción
cirugia_id, recurso_humano_id	FK	UNIQUE(cirugia, recurso, rol, fase).
rol	varchar(20)	Rol desempeñado en esta cirugía.
fase	varchar(12) = 'quirurgica'	pre | quirurgica | post.
hora_inicio, hora_fin	datetime NULL	Entrada y salida; los minutos se derivan.
minutos_participacion	smallint	Minutos costeados.
costo_mensual_registrado	decimal(14,2) NULL	Snapshot del costo mensual (con o sin indirectos según la categoría CIF activa).

8.2.14 consumo_insumos

Insumos consumidos con precio congelado.

Columna	Tipo	Descripción
cirugia_id, insumo_id	FK	UNIQUE(cirugia, insumo, fase).
fase	varchar(12) = 'quirurgica'	pre | quirurgica | post.
cantidad	decimal(10,2)	Cantidad consumida.
costo_unitario_registrado, costo_total	decimal(14,2)	Snapshot del precio y total.

8.2.15 cirugia_equipo_medico

Equipos usados en la cirugía (N:M).

Columna	Tipo	Descripción
cirugia_id, equipo_medico_id	FK	UNIQUE por par.
minutos_uso	smallint	Minutos de uso.
costo_hora_registrado	decimal(14,2) NULL	Snapshot de la tarifa.

8.2.16 costos_cirugia

Resultado del motor TDABC (1:1 con la cirugía).

Columna	Tipo	Descripción
cirugia_id	FK UNIQUE cascade	Una ficha de costo por cirugía; recostear la reemplaza.
hospital_id	FK	Tenant.
costo_recurso_humano, costo_sala, costo_equipos, costo_insumos	decimal(14,2)	Componentes directos.
costo_directo, costo_indirecto, costo_total	decimal(14,2)	Totales.
detalle	json	Desglose línea a línea, por fase y por bolsa (ver §7.6).
calculado_en	timestamp	Momento del cálculo.

8.2.17 cirugia_concepto_indirecto

Línea auditable de cada bolsa CIF aplicada a la cirugía.

Columna	Tipo	Descripción
cirugia_id, concepto_costo_indirecto_id, hospital_id	FK	UNIQUE(cirugia, concepto); índice (hospital, categoría).
nombre_registrado, categoria_registrada, base_asignacion_registrada	varchar	Snapshot descriptivo del concepto.
monto_mensual_registrado, porcentaje_registrado	decimal NULL	Parámetro de la bolsa en ese momento.
denominador_registrado	int NULL	Capacidad sumada usada como denominador.
tasa_registrada	decimal(16,6)	monto ÷ denominador (o porcentaje).
unidades_aplicadas	decimal(12,2)	Minutos o costo directo.
monto_asignado	decimal(14,2)	tasa × unidades, cuadrado por mayor resto.

8.2.18 resultados_clinicos

Desenlace clínico de la cirugía (1:1).

Columna	Tipo	Descripción
cirugia_id, hospital_id	FK	UNIQUE cirugia_id.
complicacion_intraoperatoria, descripcion_complicacion_intra	boolean, text	Complicación en sala.
complicacion_postoperatoria, descripcion_complicacion_post	boolean, text	Complicación posterior.
dias_estancia	smallint = 0	Estancia hospitalaria.
reingreso_30_dias, mortalidad	boolean	Indicadores de resultado.

8.2.19 facturaciones

Facturación de la cirugía (1:1): base de los KPIs de margen, glosas y recaudo.

Columna	Tipo	Descripción
cirugia_id, hospital_id	FK	UNIQUE cirugia_id.
valor_facturado	decimal(14,2)	Facturado a la EPS.
valor_glosado, valor_recaudado	decimal(14,2) = 0	Glosas y recaudo.
tarifa_referencia_soat	decimal(14,2) NULL	Tarifa de referencia.
fecha_facturacion	date NULL	Fecha.

8.2.20 alertas_sobrecosto

Alerta generada por DetectorSobrecostos (1:1 con la cirugía).

Columna	Tipo	Descripción
hospital_id, cirugia_id (UNIQUE), procedimiento_quirurgico_id	FK	Índices (hospital, estado) y (procedimiento, causa).
costo_total, costo_esperado, exceso, exceso_pct	decimal	Costo, mediana del baseline, exceso absoluto y relativo.
z	decimal(8,3) NULL	z-score cuando hay desviación.
criterios	json	Criterios disparados (tukey, z).
n_baseline	int	Tamaño del baseline (≥ 5).
atribucion	json	Exceso por componente.
componente_dominante	varchar(20)	recurso_humano | sala | equipos | insumos | indirecto.
estado	varchar(12) = 'pendiente'	pendiente | revisada | descartada.
causa, causa_detalle	varchar(40), text NULL	CausaSobrecosto y detalle libre.
revisado_por, revisado_en, detectado_en	FK users, timestamps	Trazabilidad de la revisión.

8.2.21 registros_actividad

Bitácora de auditoría escrita por el trait Auditable.

Columna	Tipo	Descripción
user_id, hospital_id	FK NULL (nullOnDelete)	Quién y en qué hospital.
accion, descripcion	varchar	creó / actualizó / eliminó + descripción legible.
auditable_type, auditable_id	morph NULL	Entidad afectada.
ip	varchar(45) NULL	Dirección IP.
created_at	timestamp (índice)	Fecha.

8.3 Enumeraciones del dominio

Los valores permitidos de las columnas de tipo texto se centralizan en app/Enums/ (enums PHP 8 respaldados por string) y se validan con Rule::in(Enum::values()).

Enum	Valores	Columna(s)
RolUsuario	super_admin, admin_hospital, digitador	users.role
RolQuirurgico	cirujano, ayudante, anestesiologo, instrumentador, circulante	recursos_humanos.rol, miembros_equipo_quirurgico.rol, plantilla_personal.rol
EstadoCirugia	programada, en_proceso, en_recuperacion, realizada, cancelada	cirugias.estado
TipoCirugia	programada, urgencia	cirugias.tipo
FaseCiclo	pre, quirurgica, post	miembros_equipo_quirurgico.fase, consumo_insumos.fase, plantilla_*.fase
CategoriaInsumo	medicamento, dispositivo, material	insumos.categoria
NivelComplejidad	baja, media, alta	procedimientos_quirurgicos.complejidad, hospitales.nivel_complejidad
NivelConfiabilidad	medido, estimado, supuesto	*.nivel_confiabilidad

Enum	Valores	Columna(s)
`Regimen`	contributivo, subsidiado, especial, no_asegurado	`pacientes.regimen`
`CategoriaCif`	infraestructura, depreciacion_equipos, personal_indirecto, administracion, servicios_generales	`conceptos_costo_indirecto.categoria`
`BaseAsignacionCif`	minuto_quirofano, minuto_personal, porcentaje_directo	`conceptos_costo_indirecto.base_asignacion`
`OrigenComponente`	digitado, derivado_de_cif	`hospitales.origen_*`
`ComponenteCosto`	recurso_humano, sala, equipos, insumos, indirecto	`alertas_sobrecosto.componente_dominante`
`EstadoAlerta`	pendiente, revisada, descartada	`alertas_sobrecosto.estado`
`CausaSobrecosto`	consumo_excesivo_insumos, tiempo_quirurgico_prolongado, retraso_alistamiento, equipo_adicional, personal_adicional, complicacion_clinica, caso_complejo_justificado, error_de_registro, otra	`alertas_sobrecosto.causa`

8.4 Datos de demostración

`php artisan migrate:fresh --seed` ejecuta `DatabaseSeeder` → `DemoSeeder` (`HospitalSeeder`, `UsuarioSeeder`, `CatalogoQuirurgicoSeeder`, `PacienteSeeder`, `CirugiaSeeder`, `CosteoSeeder`). Crea dos hospitales ficticios marcados `[SEMILLA]`: **Hospital San Rafael de Maicao** (dataset completo, ~38 cirugías; usuario `admin@sanrafael.test`) y **Hospital Nuestra Señora de Riohacha** (dataset pequeño, factor indirecto 12 %; `admin@riohacha.test`), contraseña `password`. La cesárea de referencia da exactamente $520.000. Los códigos CUPS/ATC son de ejemplo.

9. Estructura del proyecto

Estructura estándar de Laravel con el frontend en `resources/js`. Se resaltan los directorios propios del dominio.

```
app-costeo/
├── app/
│   ├── Console/Commands/        DetectarSobrecostos, GenerarPlantillasCommand
│   ├── Enums/                   15 enums del dominio (roles, estados, fases, CIF…)
│   ├── Exceptions/              CapacidadCifNoDisponibleException, CirugiaNoCosteableException,
│   │                            SolapeDeCostoIndirectoException
│   ├── Http/
│   │   ├── Controllers/         Api/V1, Cirugias, Costeo, Pacientes, Parametros, Settings,
│   │   │                        Concerns/FiltraListado + Dashboard*, Digitador, Exportacion,
│   │   │                        Historial, HospitalActivo
│   │   ├── Middleware/          SetHospitalContext, EnsureRole, EnsureUsuarioActivo,
│   │   │                        HandleInertiaRequests, HandleAppearance
│   │   └── Requests/            30 Form Requests (+ Concerns/ReglasTrazabilidad,
│   │                            ReglasConceptoCostoIndirecto; Settings/*)
│   ├── Models/                  22 modelos; Concerns/{BelongsToHospital, Auditable};
│   │                            Scopes/HospitalScope
│   ├── Providers/               AppServiceProvider (gates), FortifyServiceProvider
│   ├── Services/
│   │   ├── Cirugias/            RegistrarCirugia, ActualizarCirugia, PresentarCirugiaDetalle
│   │   ├── Costing/             TdabcCostingService, AsignadorCif, ActivarCategoriaCif,
│   │   │                        DesactivarCategoriaCif, OutlierDetector, DetectorSobrecostos
│   │   ├── Indicators/          KpiService, PersonalCosteoService
│   │   └── Plantillas/          GeneradorPlantilla
│   └── Support/                 HospitalContext, Estadistica, Periodo, ExportadorCsv
├── bootstrap/app.php            registro de middleware (alias rol, hospital.contexto)
├── config/                      configuración de Laravel, Fortify, Inertia
├── database/
│   ├── factories/               16 factories
│   ├── migrations/              35 migraciones
│   └── seeders/                 DemoSeeder + 6 seeders temáticos
├── docs/                        cif-implementacion.md, cif-vacios-diseno.md
├── cerebro/                     vault Obsidian: resumen, backlog, decisiones, bitácora, agentes, dominio
├── resources/
│   ├── css/app.css              Tailwind 4 + tema
│   ├── js/
│   │   ├── actions/ · routes/ · wayfinder/   rutas y controladores tipados generados
│   │   ├── components/          ui/ (primitivas), cirugias/, costeo/, pacientes/, parametros/, comunes
│   │   ├── hooks/               use-appearance, use-flash-toast, use-two-factor-auth…
│   │   ├── layouts/             app-layout (sidebar), auth-layout, settings
│   │   ├── lib/                 fases.ts, filtros.ts, formato.ts, graficas.ts, utils.ts
│   │   ├── pages/               54 páginas Inertia
│   │   ├── types/               contratos TypeScript (cirugias, costeo, parametros…)
│   │   └── app.tsx · ssr.tsx    arranque de Inertia
│   └── views/app.blade.php      plantilla raíz
├── routes/                      web.php (dominio + API v1), settings.php, console.php
├── tests/                       Unit/ y Feature/ (299 pruebas)
├── .github/workflows/           lint.yml, tests.yml
├── .claude/launch.json          servidores locales (app-costeo :8801, vite :5173)
├── CLAUDE.md                    guía de trabajo del proyecto y del cerebro
├── composer.json · package.json · vite.config.ts · tsconfig.json · phpstan.neon · phpunit.xml
└── README.md
```

10. Archivos principales y su funcionalidad

Esta sección describe los archivos que concentran la lógica del sistema. Se omiten los generados automáticamente (Wayfinder) y los del starter kit sin modificaciones de dominio.

10.1 Modelos (app/Models)

Archivo	Responsabilidad
`Concerns/BelongsToHospital.php`	Trait multi-tenant: aplica `HospitalScope` y asigna `hospital_id` al crear.
`Scopes/HospitalScope.php`	Scope global que filtra toda consulta por `HospitalContext::id()`.
`Concerns/Auditable.php`	Registra en `registros_actividad` toda creación, actualización y eliminación con el usuario en sesión (los seeders y comandos no auditan).
`Hospital.php`	Capacidades: `minutosDisponiblesMes()`, `capacidadQuirofanosMes()`, `capacidadPersonalQuirurgicoMes()`, `denominadoresCif()`, `minutosDisponiblesEntre()`; orígenes de componentes (`origenDe()`); relaciones con todo el catálogo. `origen_*` fuera de $fillable.
`User.php`	Usuario con rol y hospital: `isSuperAdmin()`, `isAdminHospital()`, `isDigitador()`; 2FA y passkeys.
`Cirugia.php`	Objeto de costo. Tiempos derivados: `duracionMinutos()`, `minutosPrequirurgico()`, `minutosQuirurgicoNeto()`, `minutosRecuperacion()`, `cicloTotalMinutos()`, `pasoDeCierre()`; `procedimientoPrincipal()`; relaciones con equipo, consumos, equipos médicos, conceptos indirectos, costo, resultado clínico y facturación; cast JSON del snapshot CIF.
`Paciente.php`	Documento cifrado (cast `encrypted`) y hash SHA-256 calculado al asignar.
`RecursoHumano.php`	`costoMensualTotal(bool $conIndirectos)`: salario + prestaciones (+ indirectos).
`ProcedimientoQuirurgico.php`	Catálogo CUPS con tiempos por fase y relaciones con las tres plantillas.
`Insumo.php`, `EquipoMedico.php`, `SalaOperatoria.php`	Catálogo con trazabilidad y estado activo.
`MiembroEquipoQuirurgico.php`, `ConsumoInsumo.php`	Pivotes con fase (cast a `FaseCiclo`) y valores congelados.
`ConceptoCostoIndirecto.php`	Bolsa CIF; `activo` fuera de $fillable; scopes de vigencia.
`CirugiaConceptoIndirecto.php`	Línea auditable del reparto CIF.
`CostoCirugia.php`	Ficha de costo con `detalle` JSON.
`AlertaSobrecosto.php`	Alerta con estado, causa, atribución y revisión.
`Facturacion.php`, `ResultadoClinico.php`	Cierre del ciclo de datos (1:1).
`PlantillaPersonal.php`, `PlantillaInsumo.php`, `PlantillaEquipo.php`	Plantilla del protocolo por procedimiento.
`RegistroActividad.php`	Entrada del historial de auditoría (morph `auditable`).

10.2 Servicios de dominio (app/Services)

Archivo	Responsabilidad
`Costing/TdabcCostingService.php`	**Motor TDABC.** `calcular(Cirugia): CostoCirugia`. Exige estado realizada; lee snapshots; calcula RRHH, sala, equipos, insumos por fase; elige la vía del indirecto (bolsas o factor); aplica exclusiones por origen; prorratea el indirecto por fase con mayor resto; escribe `costos_cirugia` y el pivote CIF en una transacción; invoca al detector de sobrecostos.
`Costing/AsignadorCif.php`	`congelar(Hospital, fecha)`: fotografía vía, orígenes, denominadores y bolsas vigentes con su tasa. `asignar(parametros, unidades)`: reparte cada bolsa por su inductor. `repartirPorMayorResto()` (estático) y `derivadoDeCif()`.
`Costing/ActivarCategoriaCif.php`	Único escritor de `activo` + `origen_*`: enciende las bolsas de una categoría y marca el componente directo equivalente en una transacción. `componentesDigitados()` nombra los registros con valor > 0 para el diálogo de confirmación. Lanza `CapacidadCifNoDisponibleException` si no hay capacidad.
`Costing/DesactivarCategoriaCif.php`	Operación inversa: apaga las bolsas y devuelve el origen a digitado.
`Costing/OutlierDetector.php`	Outliers por procedimiento con z-score (|z| > 3) y Tukey (1,5·IQR); ventana temporal opcional.
`Costing/DetectorSobrecostos.php`	`evaluar(CostoCirugia): ?AlertaSobrecosto`. Baseline ≥ 5 cirugías del mismo procedimiento; exceso ≥ 10 % sobre la mediana; atribución del exceso por componente; crea o actualiza la alerta.
`Indicators/KpiService.php`	KPIs Donabedian con ventana temporal (ver §7.8). Consultas acotadas por el scope y a cirugías realizadas con costo.
`Indicators/PersonalCosteoService.php`	Costo propio e inducido por persona, índice ajustado por case-mix, ranking, ficha, desgloses, por procedimiento e historial paginado. Usa SQL agregado con filtro explícito por hospital.
`Cirugias/RegistrarCirugia.php`	Registro transaccional de la cirugía con procedimientos, equipo, consumos y equipos médicos; congela todas las tarifas y el snapshot CIF (`AsignadorCif::congelar`). Compartido por la UI y la API.
`Cirugias/ActualizarCirugia.php`	Corrección transaccional que conserva las tarifas congeladas de lo existente; solo lo nuevo toma tarifa vigente; recongela la sala solo si cambió.
`Cirugias/PresentarCirugiaDetalle.php`	Serializa la cirugía con todas sus relaciones para las vistas de detalle (registro y costeo) con un único formato.
`Plantillas/GeneradorPlantilla.php`	Deduce la plantilla del histórico: una línea entra si aparece en ≥ 50 % de las cirugías, como opcional si ≥ 25 %. `proponer()` y `generar()`.

10.3 Capa HTTP (app/Http)

Archivo	Responsabilidad
`Middleware/SetHospitalContext.php`	Alias `hospital.contexto`. Fija el tenant: admin_hospital siempre a su hospital (fail-closed); super_admin sin scope salvo selección en sesión.
`Middleware/EnsureRole.php`	Alias `rol:a,b`. 403 si el rol no está permitido.
`Middleware/EnsureUsuarioActivo.php`	Cierra la sesión de usuarios desactivados.
`Middleware/HandleInertiaRequests.php`	Props compartidas: auth (usuario, rol), hospital activo y lista, flash, apariencia.
`Controllers/Parametros/*.php (9)`	CRUD de cada catálogo, configuración del hospital, plantilla del protocolo (edit/update/sugerir), conceptos CIF (activar/desactivar). Usan `FiltraListado` para búsqueda y paginación.
`Controllers/Cirugias/CirugiaController.php`	index, create, store, show, edit, update, cerrar, destroy, calcular. Delegan en los servicios de Cirugias y en el motor.
`Controllers/Cirugias/{Facturacion,ResultadoClinico,Paciente}Controller.php`	Captura de facturación, resultado clínico y alta rápida de paciente desde el registro.
`Controllers/Costeo/ProcedimientoCosteoController.php`	Explorador: catálogo → cirugías → detalle costeado.
`Controllers/Costeo/PersonalCosteoController.php`	Ranking y ficha por persona.
`Controllers/Costeo/AlertaSobrecostoController.php`	Bandeja y revisión de alertas.
`Controllers/DashboardCosteoController.php`	index, componentes, outliers, rentabilidad, variabilidad (con `Periodo::desdeRequest`).
`Controllers/DashboardController.php`	Dashboard consolidado del super_admin.
`Controllers/HospitalActivoController.php`	Switcher de hospital (gate `elegir-hospital`).
`Controllers/HistorialController.php`	Listado paginado del historial de auditoría.
`Controllers/DigitadorController.php`	Gestión de digitadores del hospital.
`Controllers/ExportacionController.php`	CSV de cirugías, indicadores y pendientes.
`Controllers/Pacientes/PacienteController.php`	Padrón de pacientes.
`Controllers/Api/V1/*.php (6)`	KpiController (8 endpoints JSON) y captura JSON de pacientes, insumos, procedimientos, recursos humanos y cirugías (+ calcular-costo).
`Controllers/Concerns/FiltraListado.php`	Filtros comunes de listados: texto, estado, fechas, orden y paginación.
`Requests/StoreCirugiaRequest.php, UpdateCirugiaRequest.php`	Validación completa del registro: paciente/sala/procedimientos/insumos/equipos **existentes en el hospital**, CIE-10, coherencia cronológica de todas las marcas (validador `after`), roles y fases válidos, minutos 1–1440, cantidades > 0.
`Requests/Store*/Update* de parámetros`	Validación de cada catálogo: CUPS de 6 dígitos único por hospital, ATC obligatorio en medicamentos, montos ≥ 0, trazabilidad (`ReglasTrazabilidad`).
`Requests/*ConceptoCostoIndirectoRequest.php + Concerns/ReglasConceptoCostoIndirecto.php`	Monto vs. porcentaje excluyentes según la base; vigencias sin solape; bloqueo de edición de campos sensibles en conceptos activos.
`Requests/CerrarCirugiaRequest, GuardarFacturacionRequest, GuardarResultadoClinicoRequest, RevisarAlertaSobrecostoRequest, ActivarCategoriaCifRequest, GuardarPlantillaProcedimientoRequest, StoreDigitadorRequest`	Validaciones de las operaciones específicas.

10.4 Soporte, excepciones y proveedores

Archivo	Responsabilidad
`Support/HospitalContext.php`	`set()`, `id()`, `clear()`: resuelve el hospital activo (usuario, sesión o fijado explícitamente por seeders/tests).
`Support/Estadistica.php`	`media`, `desviacionEstandar`, `coeficienteVariacion`, `percentil`.
`Support/Periodo.php`	Ventana temporal: `desdeRequest()`, `ultimosMeses()`, `vacio()`, `etiqueta()`.
`Support/ExportadorCsv.php`	Descarga CSV en streaming con nombre de archivo fechado.
`Exceptions/CirugiaNoCosteableException.php`	Se intenta costear una cirugía que no está realizada.
`Exceptions/CapacidadCifNoDisponibleException.php`	Activar una bolsa por minuto sin salas o personal activos (evita división por cero); nombra el arreglo.
`Exceptions/SolapeDeCostoIndirectoException.php`	Dos vigencias del mismo concepto se pisan.
`Providers/AppServiceProvider.php`	Gates: `elegir-hospital` (super_admin), `operar-hospital` (hay hospital activo), `corregir-cirugia` (admin: cualquiera de su hospital; digitador: solo las que registró).
`Providers/FortifyServiceProvider.php`	Vistas Inertia de autenticación, 2FA, passkeys.
`bootstrap/app.php`	Registro de middleware web y alias `rol` / `hospital.contexto`.

10.5 Frontend (resources/js)

Archivo / carpeta	Responsabilidad
`app.tsx, ssr.tsx`	Arranque de Inertia + React (resolución de páginas, tema, toasts).
`layouts/app-layout.tsx, layouts/app/*`	Layout principal con sidebar (`app-sidebar.tsx`, `nav-main.tsx`) que muestra los módulos según el rol, badge del hospital activo (`contexto-hospital-badge.tsx`) y switcher (`hospital-switcher.tsx`).
`pages/cirugias/*`	inicio (digitador), index (listado con filtros), create/edit (formulario completo), show (detalle costeado).
`components/cirugias/formulario-cirugia.tsx`	Formulario del registro: paciente, procedimientos, sala, marcas de tiempo, equipo por fase, consumos, equipos médicos; prellenado por plantilla.
`components/cirugias/estimacion-costo.tsx`	Estimación de costo en vivo con las tarifas vigentes mientras se digita.
`components/cirugias/desglose-costo.tsx`	Desglose por componente, fase y bolsa CIF de una cirugía costeada.
`components/cirugias/{cerrar-cirugia-modal, nuevo-paciente-modal, facturacion-card, resultado-clinico-card}.tsx`	Cierre, alta rápida de paciente, captura de facturación y de resultado clínico.
`pages/costeo/*`	index (resumen), componentes, outliers, rentabilidad, variabilidad, alertas, procedimientos/{index, show, cirugia}, personal/{index, show}.
`components/costeo/*`	kpi-card, selector-periodo, tendencia-costos, boxplot-costos, dispersion-procedimiento, composicion-cirugia, indice-badge, tabla-responsive, encabezado-costeo.
`pages/parametros/*`	index (menú), hospital (configuración), y por catálogo index/create/edit; procedimientos/plantilla; costos-indirectos.
`components/parametros/forms/*.tsx`	Formularios de cada catálogo, plantilla del protocolo y concepto CIF (muestra la tasa por minuto en vivo).
`components/parametros/activacion-categorias-cif.tsx`	Panel de encendido/apagado de categorías CIF con diálogo que nombra los conflictos.
`components/parametros/{campos-trazabilidad, nivel-confiabilidad-badge, modal-formulario, confirmar-eliminacion, encabezado-listado, paginacion}.tsx`	Piezas comunes de los CRUD.
`pages/pacientes/index.tsx, components/pacientes/*`	Padrón y edición de pacientes.
`pages/dashboard.tsx, historial/index.tsx, digitadores/*`	Dashboard consolidado, historial de auditoría, gestión de digitadores.
`pages/auth/*, pages/settings/*`	Autenticación y ajustes del starter kit (2FA, passkeys, apariencia).
`components/ui/*`	Primitivas de interfaz (botón, input, select, dialog, table, tooltip…) sobre Radix UI.
`components/{filtros-listado, boton-exportar, buscador-select, breadcrumbs, alert-error}.tsx`	Filtros comunes, exportación CSV, selector con búsqueda, migas, errores.
`lib/fases.ts`	Etiquetas y orden de las fases del ciclo.
`lib/formato.ts`	Formato de moneda COP, porcentajes, minutos y fechas.
`lib/graficas.ts`	Paleta y utilidades de Recharts.
`lib/filtros.ts, lib/utils.ts`	Serialización de filtros a query string; `cn()`.
`hooks/*`	use-flash-toast (mensajes flash → sonner), use-appearance (tema), use-two-factor-auth, use-mobile, use-initials, use-clipboard, use-current-url.
`types/*.ts`	Contratos TypeScript de las props: cirugias, costeo, parametros, pacientes, auth, navigation, ui.
`actions/, routes/, wayfinder/`	Generados por Wayfinder: funciones tipadas para cada ruta y método de controlador.

11. Rutas y API

Definidas en `routes/web.php` (dominio y API v1) y `routes/settings.php` (perfil y seguridad). Todas exigen sesión autenticada y correo verificado; los grupos aplican `rol:` y `hospital.contexto`.

11.1 Rutas web

Método	Ruta	Acción	Roles
GET	`/`	Redirige según rol: digitador → cirugías; admin_hospital → parámetros; super_admin → dashboard	todos
GET	`/dashboard`	Dashboard consolidado	super_admin
POST	`/hospital-activo`	Cambiar hospital activo	super_admin
GET	`/historial`	Historial de auditoría	super_admin, admin_hospital
GET/POST/PATCH	`/digitadores, /digitadores/create, /digitadores/{usuario}/activo`	Gestión de digitadores	admin_hospital
GET	`/parametros`	Menú de parámetros	super_admin, admin_hospital
GET/PUT	`/parametros/hospital`	Configuración del hospital	ídem
resource	`/parametros/{recursos-humanos | insumos | equipos-medicos | salas-operatorias | procedimientos | costos-indirectos}`	index, create, store, edit, update, destroy	ídem
POST	`/parametros/costos-indirectos/{activar | desactivar}`	Encender / apagar categoría CIF	ídem
GET/PUT/POST	`/parametros/procedimientos/{procedimiento}/plantilla[/sugerir]`	Plantilla del protocolo y sugerencia	ídem
GET	`/cirugias, /cirugias/create`	Listado y nuevo registro	todos
POST	`/cirugias, /cirugias/pacientes`	Registrar cirugía; alta rápida de paciente	todos
GET/PUT/PATCH	`/cirugias/{cirugia}/edit, /cirugias/{cirugia}, /cirugias/{cirugia}/cerrar`	Corregir y cerrar (gate corregir-cirugia)	todos
GET/POST/DELETE	`/cirugias/{cirugia}, …/calcular-costo, …`	Detalle, recostear, eliminar	super_admin, admin_hospital
POST	`/cirugias/{cirugia}/{facturacion | resultado-clinico}`	Cierre del ciclo de datos	ídem
GET	`/exportar/{cirugias | indicadores | pendientes}`	CSV con filtros	ídem
GET/POST/PUT/DELETE	`/pacientes[/{paciente}]`	Padrón de pacientes	ídem
GET	`/costeo, /costeo/{componentes | outliers | rentabilidad | variabilidad}`	Dashboards	ídem
GET	`/costeo/procedimientos[/{procedimiento}[/cirugias/{cirugia}]]`	Explorador por procedimiento	ídem
GET	`/costeo/personal[/{personal}]`	Costeo por persona	ídem
GET/PATCH	`/costeo/alertas[/{alerta}]`	Bandeja y revisión de alertas	ídem
GET/PUT/DELETE…	`/settings/{profile | password | two-factor | passkeys | appearance}`	Ajustes del usuario (routes/settings.php)	todos

11.2 API v1 (JSON, sesión web)

Prefijo `/api/v1`, middleware `auth`, `rol:super_admin,admin_hospital`, `hospital.contexto`. La consumen los dashboards y las pruebas.

Método	Ruta	KPI / acción	Dimensión
GET	`kpis/costos`	Costo promedio por cirugía y por procedimiento	Proceso
GET	`kpis/variabilidad`	Coeficiente de variación por procedimiento	Proceso
GET	`kpis/margen`	Costo real vs. tarifa facturada y SOAT −25 %	Resultado
GET	`kpis/utilizacion-salas?mes=YYYY-MM`	Minutos operados ÷ disponibles por sala	Estructura
GET	`kpis/glosas-recaudo`	Tasas de glosas y recaudo	Resultado
GET	`kpis/completitud`	% de cirugías con equipo / insumos / costo / resultado / factura	Supervisión
GET	`kpis/outliers`	Costos atípicos por procedimiento (z + IQR)	—
GET	`kpis/componentes`	Costo promedio por componente TDABC	Proceso
POST	`pacientes, insumos, procedimientos, recursos-humanos`	Captura con validación (mismos Form Requests que la web)	—
POST	`cirugias`	Cirugía + procedimientos + equipo + consumos (transaccional)	—
POST	`cirugias/{id}/calcular-costo`	Ejecuta el motor TDABC	—

Todos los endpoints de KPIs aceptan los parámetros de periodo (`desde`, `hasta`) que interpreta `Periodo::desdeRequest()`.

12. Seguridad, roles y protección de datos

12.1 Autenticación

Laravel Fortify con sesión de servidor: inicio de sesión con correo y contraseña, verificación obligatoria de correo (`verified`), recuperación de contraseña, confirmación de contraseña para acciones sensibles, **2FA TOTP** con códigos de recuperación y **passkeys** (WebAuthn). Las contraseñas se almacenan con hash bcrypt; CSRF protegido por el framework.

12.2 Autorización por roles

Rol	Alcance	Puede	No puede
super_admin	Todos los hospitales	Elegir hospital activo, dashboard consolidado, historial, y con un hospital seleccionado todo lo del admin_hospital	Operar sin seleccionar hospital (gate operar-hospital)
admin_hospital	Su hospital	Parámetros, cirugías (todo), costeo, pacientes, exportaciones, gestión de digitadores, historial	Ver otros hospitales; su hospital_id es obligatorio (fail-closed)
digitador	Su hospital, sus registros	Registrar cirugías, alta rápida de pacientes, corregir y cerrar las cirugías que él capturó (gate corregir-cirugia)	Ver el histórico del hospital, el detalle costeado, eliminar, parámetros, costeo

Los usuarios pueden desactivarse (`users.activo`); `EnsureUsuarioActivo` cierra la sesión de un usuario dado de baja. La navegación del frontend oculta los módulos no permitidos, pero la autorización real ocurre en el servidor (middleware y gates).

12.3 Aislamiento de datos

- Scope global por hospital en todos los modelos de dominio; 404 para recursos de otro tenant.
- Reglas de validación `exists` acotadas al hospital del usuario.
- Claves foráneas con `restrictOnDelete` hacia el catálogo (no se puede borrar lo referenciado) y `cascadeOnDelete` de la cirugía hacia sus pivotes.
- Pruebas `TenantIsolationTest`, `SuperAdminTest`, `DigitadorTest` y `DashboardPorRolTest` verifican el aislamiento y los permisos.

12.4 Protección de datos personales (Ley 1581/2012)

- El documento del paciente se guarda cifrado con la clave de la aplicación (`APP_KEY`) y se busca por su hash SHA-256.
- Los indicadores y exportaciones de KPIs devuelven agregados, no datos identificables.
- El historial de auditoría registra quién hizo qué, cuándo y desde qué IP.
- El repositorio público no contiene datos reales de pacientes; los seeders son ficticios.

12.5 Integridad del costeo

- Los parámetros CIF activos y los orígenes solo se escriben desde `ActivarCategoriaCif` / `DesactivarCategoriaCif` (fuera de `$fillable`).
- Un concepto CIF activo no se puede borrar ni cambiar de monto, porcentaje, base, categoría o inicio de vigencia; se cierra su vigencia y se abre otro.
- Solo cirugías **realizadas** se costean; el costo se calcula siempre desde los valores congelados.

13. Calidad, pruebas e integración continua

13.1 Suite de pruebas

`php artisan test` → **299 pruebas, 1.521 aserciones, todas pasando** (verificado el 2026-09-19, 10 s con SQLite en memoria). Las páginas Inertia requieren `npm run build` previo porque se resuelven por el manifiesto de Vite.

Archivo de prueba	Qué verifica
`Unit/TdabcCostingServiceTest`	**Caso obligatorio**: cesárea = $520.000; derivación del costo/minuto; factor de indirectos; recálculo idempotente.
`Unit/DetectorSobrecostosTest`	Baseline mínimo, umbral de exceso, atribución por componente, componente dominante.
`Unit/GeneradorPlantillaTest`	Umbrales 50 % / 25 % de la sugerencia de plantilla.
`Feature/Costeo/MotorCifTest (14)`	Vía factor sin bolsas; bolsa por minuto de quirófano ($28.080.000 ÷ 56.160 × 120 = $60.000); tasa baja con más salas; bolsa por personal; porcentaje sobre el directo excluido; exclusiones de sala/equipos/personal; factor ignorado con bolsas; Σ líneas = indirecto; Σ por fase = indirecto; recostear reemplaza líneas; cirugía previa conserva su vía; bolsa vencida; activación sin capacidad.
`Feature/Costeo/ActivarCategoriaCifTest (12)`	Invariante anti-doble-conteo, confirmación, rutas de activación/desactivación.
`Feature/Parametros/ConceptoCostoIndirectoTest (16)`	CRUD, validaciones excluyentes, vigencias sin solape, guardas de edición de conceptos activos.
`Feature/Costeo/CapacidadEfectivaTest`	minutos_efectivos_hora cambia la capacidad y las tarifas por minuto; se congela en la cirugía.
`Feature/Costeo/AlertasSobrecostoTest`, `NotificacionSobrecostoTest`	Generación, bandeja, revisión con causa, notificación.
`Feature/Costeo/PeriodoYFiltrosTest`, `ProcedimientosCosteoTest`, `PersonalCosteoTest`	Ventanas temporales, explorador por procedimiento, costeo por persona (totales cuadran con costos_cirugia).
`Feature/Cirugias/CapturaCicloCompletoTest`, `FasesDelCicloTest`, `CorregirCirugiaTest`, `CirugiaWebTest`, `ReglasCosteoTest`	Ciclo pre → quirúrgica → post, estados, marcas de tiempo, corrección sin reescribir tarifas, cierre, reglas de costeo (solo realizadas).
`Feature/Parametros/*CrudTest`, `HospitalConfiguracionTest`, `PlantillaProcedimientoTest`	CRUD de cada catálogo con trazabilidad, configuración del hospital, plantilla.
`Feature/TenantIsolationTest`, `SuperAdminTest`, `DigitadorTest`, `DashboardPorRolTest`	Aislamiento por hospital, switcher, permisos del digitador, vistas por rol.
`Feature/KpiEndpointsTest`	Valores exactos de cada KPI y detección de outliers.
`Feature/ValidacionesTest`	Consistencia temporal, CIE-10, CUPS, ATC, unicidad de paciente por hash, integridad referencial entre hospitales.
`Feature/DemoSeederTest`, `SeedersCatalogoTest`	El seeder reproduce la cesárea y separa los datos de los dos hospitales.
`Feature/HistorialTest`	El trait Auditable escribe el historial.
`Feature/Auth/*`, `Settings/*`	Autenticación, 2FA, perfil y seguridad (starter kit).

13.2 Análisis estático y formato

Comando	Herramienta	Alcance
`composer types:check`	PHPStan / Larastan nivel 7	app/, bootstrap/app.php, config/, database/
`composer lint / lint:check`	Laravel Pint	Formato PHP
`npm run lint / lint:check`	ESLint 9 (React, hooks, import, TypeScript)	resources/js
`npm run format / format:check`	Prettier + plugin Tailwind	resources/
`npm run types:check`	tsc --noEmit	TypeScript estricto
`composer ci:check`	Todo lo anterior + pruebas	Verificación completa local

13.3 Integración continua

- `.github/workflows/tests.yml`: en cada push/PR instala Node y Composer, copia `.env.example`, genera la clave, construye los assets con Vite y ejecuta php `artisan test` en una matriz de **PHP 8.3, 8.4 y 8.5**.
- `.github/workflows/lint.yml`: Pint, Prettier y ESLint con PHP 8.4.

14. Instalación, configuración y despliegue

14.1 Requisitos

- PHP ≥ 8.3 con extensiones estándar de Laravel (mbstring, openssl, pdo_mysql, bcmath, gd/intl según necesidad).
- Composer 2.
- Node.js ≥ 20 y npm.
- MySQL 8 (producción y desarrollo). SQLite solo para pruebas.

14.2 Puesta en marcha local

```
git clone https://github.com/ZapataMor/app-costeo.git && cd app-costeo
composer install
npm install
cp .env.example .env
php artisan key:generate
# Configurar la conexión MySQL en .env (DB_CONNECTION=mysql, DB_HOST, DB_PORT, DB_DATABASE, DB_USERNAME, DB_PASSWORD)
php artisan migrate:fresh --seed          # esquema + dos hospitales de demostración
npm run build                             # o `npm run dev` para HMR
php artisan serve --port=8801             # http://localhost:8801
```

Alternativa: `composer setup` (instala, crea .env, genera clave, migra, instala npm y construye) y `composer dev` (servidor + cola + Vite en paralelo). En el entorno de desarrollo del proyecto existen dos servidores definidos en `.claude/launch.json`: `app-costeo` (puerto 8801) y `vite` (puerto 5173).

14.3 Variables de entorno relevantes

Variable	Uso
`APP_KEY`	Clave de cifrado: cifra el documento del paciente y las sesiones. **No cambiarla** tras tener datos: los documentos cifrados dejarían de poder leerse.
`APP_URL, APP_ENV, APP_DEBUG`	URL pública, entorno (`production`) y depuración (desactivar en producción).
`DB_*`	Conexión MySQL 8.
`SESSION_DRIVER, CACHE_STORE, QUEUE_CONNECTION`	Por defecto database; en producción pueden usarse redis.
`MAIL_*`	Correo para verificación de cuenta y recuperación de contraseña.

14.4 Despliegue en producción (lineamientos)

- Servidor web (Nginx/Apache) apuntando a `public/`; PHP-FPM 8.3+.
- `composer install --no-dev --optimize-autoloader`, `npm ci && npm run build`, `php artisan migrate --force`, `php artisan config:cache route:cache view:cache`.
- Un *worker* de cola (`php artisan queue:work`) si se usan notificaciones en segundo plano; el *scheduler* para ejecutar `costeo:detectar-sobrecostos` periódicamente si se desea.
- HTTPS obligatorio (passkeys y cookies seguras lo requieren).
- Copias de seguridad de la base de datos y custodia de `APP_KEY`.
- A la fecha **no hay despliegue en producción** ni datos reales de hospitales; los seeders son ficticios.

14.5 Puesta en marcha del costeo en un hospital

1) Configurar la capacidad práctica en *Parámetros → Hospital* (Kaplan & Anderson recomiendan ≈ 48 min efectivos por hora). 2) Cargar el catálogo: salas, personal, insumos, equipos, procedimientos, con su fuente y confiabilidad. 3) Registrar las bolsas CIF y activar sus categorías (o fijar un factor indirecto). 4) Definir o sugerir la plantilla de cada procedimiento. 5) Crear los digitadores. 6) Registrar cirugías: cada una congela sus parámetros y, al cerrarse como realizada, se costea y evalúa automáticamente.

15. Estado actual, backlog y próximos pasos

15.1 Estado al 2026-09-19

- Capas 2 (datos) y 3 (información/BI) de la arquitectura de la tesis **completas**; capa 4 parcial (alertas de sobrecosto); capa 5 parcial (tres roles).
- Último cambio funcional: 2026-09-07, motor de asignación de bolsas CIF completo (documentado en `docs/cif-implementacion.md`).
- 299 pruebas pasando; análisis estático y formato limpios; CI verde.
- Sin despliegue en producción ni datos reales de hospitales (pendiente de confirmación con el cliente).
- Base de conocimiento del proyecto en `cerebro/` (resumen, cliente y negocio, decisiones, bitácora, backlog, agentes de análisis, fuentes de dominio).

15.2 Backlog priorizado

Lista numerada y nunca renumerada en `cerebro/backlog-mejoras.md`. P1 bloquea el valor del producto o la tesis; P2 mejora clara; P3 deseable.

#	Mejora	Prioridad
1	Alinear los umbrales de la alerta de sobrecosto con la clasificación del pipeline doctoral (Subejecución ≤ −5 %, En rango, Sobrecosto leve 5–20 %, Sobrecosto alto > 20 %).	P1
2	Reporte de capacidad no utilizada por sala y por recurso, valorada en COP (KPI propio del TDABC).	P1
3	Costo estándar (duración planificada) vs. costo real por cirugía con desviación.	P1
15	Hacer privado el repositorio o depurar el material sensible del historial.	P1
4	Horas extra: sobrecosto del 50 % en capacidad después de una hora de corte configurable.	P2
5	Factor de ineficiencia institucional configurable por hospital (1,15–1,28).	P2
6	Ecuaciones de tiempo por procedimiento (términos condicionales: complicación, conversión, reintervención).	P2
7	Punto de equilibrio quirúrgico mensual.	P2
8	Tiempos de espera del paciente derivados de las marcas de fase.	P2
9	Portal individual del cirujano con benchmarking anónimo (capa 5).	P2
10	Capa 4 — lecciones aprendidas al cerrar una alerta con causa.	P2
14	Actualizar el README (dice 61 pruebas y «Capa 1 = parámetros»).	P2
18	Indicadores operativos de la Fundamentación: turnover, cancelaciones, horas extra, esperas, latencia de registro.	P2
19	Dataset de validación con las 3.100 cirugías simuladas del pipeline doctoral.	P2
11	Captura en sala con tablet y modo offline.	P3
12	Integración con el HIS/HCE de cada hospital (archivos planos → HL7/API).	P3
13	Catálogo de insumos con código de barras / CUM, consignación, desperdicio y devoluciones.	P3
16	Portal ejecutivo consolidado multi-hospital con semáforos y exportación PDF/Excel.	P3
17	Módulo de comparación anónima entre hospitales (red departamental).	P3
20	Campos clínicos adicionales (ASA, comorbilidades, IMC, prioridad, servicios de apoyo).	P3
21	Unificar la numeración de capas en UI y docs con la de la tesis.	P3

15.3 Próximos pasos

- Confirmar con el cliente el estado del contrato y del levantamiento en los hospitales.

- Resolver las preguntas abiertas del CIF con el profesor (origen del monto mensual de cada bolsa; si el factor de ineficiencia entra al aplicativo; conciliar las tasas del pipeline con las de la cesárea de $520.000).
- Atacar el backlog P1 (umbrales, capacidad no utilizada, costo estándar vs. real).
- Capas 4 y 5: lecciones aprendidas, portales por usuario, puesta en producción y capacitación.

Anexo A. Glosario

Término	Definición
TDABC	*Time-Driven Activity-Based Costing*: costeo basado en actividades dirigido por el tiempo. El costo de un servicio es la suma de (tasa de capacidad por minuto de cada recurso × minutos usados) más los materiales.
ABC	Costeo basado en actividades: asigna los costos indirectos a los productos mediante inductores que reflejan su consumo real.
CIF	Costos indirectos de fabricación / de funcionamiento: energía, mantenimiento, depreciación, administración, servicios generales.
Bolsa CIF	Concepto de costo indirecto con monto mensual, categoría, inductor (base de asignación) y vigencia.
Inductor / base de asignación	Unidad que reparte una bolsa: minuto de quirófano, minuto de personal o porcentaje del costo directo.
Capacidad práctica	Minutos realmente productivos de un recurso (≈ 80 % de la jornada teórica). Configurable como `minutos_efectivos_hora`.
Snapshot	Copia congelada de los parámetros vigentes en el momento del registro de la cirugía; el costo histórico no cambia al actualizar el catálogo.
Mayor resto	Método de reparto de un total redondeado entre partes proporcionales de modo que la suma cuadre exactamente.
Outlier	Cirugía cuyo costo se sale del rango estadístico de su procedimiento (z-score o Tukey/IQR).
Sobrecosto	Exceso del costo de una cirugía sobre la mediana de su procedimiento, significativo estadística (> umbrales) y económicamente (≥ 10 %).
Donabedian	Marco de calidad en salud con tres dimensiones: estructura, proceso y resultado. Organiza los KPIs.
Kerschberg	Arquitectura de sistemas de gestión del conocimiento por capas (objetos, datos, información, conocimiento, presentación) adoptada por la tesis.
CUPS	Clasificación Única de Procedimientos en Salud (Colombia).
CIE-10	Clasificación Internacional de Enfermedades, 10.ª revisión.
ATC	Sistema de clasificación Anatómica, Terapéutica y Química de medicamentos.
SOAT	Tarifario de referencia colombiano; los hospitales pactan tarifas como SOAT menos un porcentaje.
EPS / IPS	Entidad Promotora de Salud (aseguradora) / Institución Prestadora de Servicios (hospital, clínica).
Glosa	Objeción de la EPS a una factura que reduce el valor reconocido.
Case-mix	Mezcla de casos; el índice ajustado por case-mix compara el costo de cada cirugía con el promedio de su propio procedimiento.
Multi-tenant	Un solo despliegue que sirve a varios hospitales con datos aislados por `hospital_id`.
Inertia.js	Protocolo que permite construir SPAs con controladores del servidor que devuelven componentes y props, sin API REST intermedia.
Fortify	Paquete de Laravel que implementa la lógica de autenticación (login, registro, 2FA, passkeys).
Wayfinder	Generador de funciones TypeScript tipadas a partir de las rutas y controladores de Laravel.

Anexo B. Referencias del proyecto

Recurso	Ubicación
Repositorio	`github.com/ZapataMor/app-costeo (público)`
Guía de trabajo del proyecto	`CLAUDE.md`
Resumen y estado	`cerebro/00-resumen.md`
Cliente, tesis y alcance	`cerebro/cliente-y-negocio.md`
Backlog de mejoras	`cerebro/backlog-mejoras.md`
Decisiones técnicas	`cerebro/decisiones/*.md`
Bitácora de cambios	`cerebro/bitacora/*.md`
Documentación del módulo CIF	`docs/cif-implementacion.md, docs/cif-vacios-diseno.md`
Conceptos de dominio	`cerebro/otros/conceptos/ (Costeo ABC y TDABC, Costos Indirectos (CIF), SGC, Glosario…)`
Fuentes bibliográficas del profesor	`cerebro/otros/fuentes/ (capítulo 5 de la tesis, guía de campos TDABC+MILP, libro ABC HSJ Maicao, Kaplan & Anderson…)`
Preguntas abiertas	`cerebro/otros/preguntas-abiertas.md`
README operativo	`README.md`